

B278015-DSM_Assignment2

May 18, 2026

```
[1]: #!/curl ipecho.net/plain
```

```
[2]: import pymssql
import pandas as pd
from openai import OpenAI
import matplotlib.pyplot as plt
```

```
[3]: #!/pip install pymssql
```

```
[4]: conn = pymssql.connect(server='dsimcolab.database.windows.net',
    ↪user='dsim_user', password='H1Uj&T$gm4^W', database='AdventureWorks2016')
```

```
[5]: client =
    ↪OpenAI(api_key="sk-proj-x0liHWbD6s6WM2dZWG41C53sTl9JvLzQMbKU31nlsFPHQlj-ChONkFOF1LUbHcrFRXT")
```

```
[6]: system = """You are an expert data analyst and SQL generator for the
    ↪AdventureWorks database.
```

Your task is to translate a user's natural language manufacturing or business
↪question into a safe, correct SQL Server query and a brief explanation.

Rules:

1. Only generate read-only SQL.
2. Allowed query types are:
 - SELECT
 - Common Table Expressions (CTEs)
 - ORDER BY
 - GROUP BY
 - HAVING
 - JOINS
 - window functions
 - recursive CTEs if needed
3. Do not generate:
 - INSERT
 - UPDATE
 - DELETE
 - DROP
 - ALTER

- TRUNCATE
 - EXEC
 - MERGE
 - stored procedures
4. The SQL dialect is Microsoft SQL Server (AdventureWorks).
 5. Prefer fully qualified table names, for example:
 Production.WorkOrder
 Production.Product
 Production.ProductInventory
 6. Use TOP for row limits, not LIMIT.
 7. If the question is ambiguous, make the smallest reasonable assumption and
 ↪state it briefly.
 8. If the question cannot be answered from the provided schema, say so clearly.
 9. Use joins only where supported by the schema provided.
 10. Return output in valid JSON only.

You must return exactly this JSON structure:

```
{
  "question_understanding": "short restatement of the task",
  "assumptions": ["assumption 1", "assumption 2"],
  "sql": "single SQL Server query as a string",
  "explanation": "brief explanation of how the query works",
  "result_shape": "what the output table should contain",
  "validation_checks": ["check 1", "check 2"],
  "plot_suggestion": {
    "recommended": true,
    "plot_type": "bar|line|pie|none",
    "x": "column name or null",
    "y": "column name or null",
    "title": "plot title or null"
  }
}
```

Additional requirements:

- The SQL must be executable as written.
 - Do not wrap SQL in markdown fences.
 - If no chart is appropriate, set plot_suggestion.plot_type to "none" and other
 ↪plot fields to null.
 - If aggregation is used, ensure all non-aggregated selected columns are
 ↪included in GROUP BY.
 - If ranking is requested, prefer ORDER BY and TOP, or window functions where
 ↪appropriate.
 - If multi-level bill-of-materials logic is requested, use a recursive CTE."
- system

[6]: 'You are an expert data analyst and SQL generator for the AdventureWorks database.\n\nYour task is to translate a user's natural language manufacturing or business question into a safe, correct SQL Server query and a brief explanation.\n\nRules:\n1. Only generate read-only SQL.\n2. Allowed query types are:\n - SELECT\n - Common Table Expressions (CTEs)\n - ORDER BY\n - GROUP BY\n - HAVING\n - JOINS\n - window functions\n - recursive CTEs if needed\n3. Do not generate:\n - INSERT\n - UPDATE\n - DELETE\n - DROP\n - ALTER\n - TRUNCATE\n - EXEC\n - MERGE\n - stored procedures\n4. The SQL dialect is Microsoft SQL Server (AdventureWorks).\n5. Prefer fully qualified table names, for example:\n Production.WorkOrder\n Production.Product\n Production.ProductInventory\n6. Use TOP for row limits, not LIMIT.\n7. If the question is ambiguous, make the smallest reasonable assumption and state it briefly.\n8. If the question cannot be answered from the provided schema, say so clearly.\n9. Use joins only where supported by the schema provided.\n10. Return output in valid JSON only.\n\nYou must return exactly this JSON structure:\n\n{\n "question_understanding": "short restatement of the task",\n "assumptions": ["assumption 1", "assumption 2"],\n "sql": "single SQL Server query as a string",\n "explanation": "brief explanation of how the query works",\n "result_shape": "what the output table should contain",\n "validation_checks": ["check 1", "check 2"],\n "plot_suggestion": {\n "recommended": true,\n "plot_type": "bar|line|pie|none",\n "x": "column name or null",\n "y": "column name or null",\n "title": "plot title or null"\n }\n}\n\nAdditional requirements:\n- The SQL must be executable as written.\n- Do not wrap SQL in markdown fences.\n- If no chart is appropriate, set plot_suggestion.plot_type to "none" and other plot fields to null.\n- If aggregation is used, ensure all non-aggregated selected columns are included in GROUP BY.\n- If ranking is requested, prefer ORDER BY and TOP, or window functions where appropriate.\n- If multi-level bill-of-materials logic is requested, use a recursive CTE.'

```
[7]: database_schema = """Production.WorkOrder
- WorkOrderID (PK)
- ProductID (FK -> Production.Product.ProductID)
- OrderQty
- StockedQty
- ScrappedQty
- StartDate
- EndDate
- DueDate
- ScrapReasonID

Production.Product
- ProductID (PK)
- Name
- ProductNumber
- MakeFlag
- FinishedGoodsFlag
```

- Color
- SafetyStockLevel
- ReorderPoint
- StandardCost
- ListPrice
- ProductSubcategoryID

Production.ProductInventory

- ProductID (FK -> Production.Product.ProductID)
- LocationID (FK -> Production.Location.LocationID)
- Shelf
- Bin
- Quantity

Production.Location

- LocationID (PK)
- Name
- CostRate
- Availability

Production.WorkOrderRouting

- WorkOrderID (FK -> Production.WorkOrder.WorkOrderID)
- ProductID
- OperationSequence
- LocationID (FK -> Production.Location.LocationID)
- ScheduledStartDate
- ScheduledEndDate
- ActualStartDate
- ActualEndDate
- PlannedCost
- ActualCost

Production.BillOfMaterials

- BillOfMaterialsID (PK)
- ProductAssemblyID
- ComponentID
- StartDate
- EndDate
- UnitMeasureCode
- BOMLevel
- PerAssemblyQty

Production.TransactionHistory

- TransactionID
- ProductID
- ReferenceOrderID
- ReferenceOrderLineID

```
- TransactionDate
- TransactionType
- Quantity
- ActualCost"""
```

```
[8]: user_question = "Which 10 products have the highest on-hand quantity in total,
↳and where is that stock located?"
```

```
[9]: user_content = f"""Database: AdventureWorks
Task: Convert the following natural language request into SQL Server SQL.

Schema summary:
{database_schema}

Business context:
- This database represents a manufacturing company.

Important table hints:
- Work orders: Production.WorkOrder + Production.Product
- Inventory: Production.ProductInventory + Production.Location + Production.
↳Product

Natural language request:
{user_question}

Output requirements:
- Return valid JSON only.
"""
```

```
[10]: message = [
    {
        "role": "system",
        "content": system
    },
    {
        "role": "user",
        "content": user_content    # MUST be a string
    }
]
message
```

```
[10]: [{'role': 'system',
        'content': 'You are an expert data analyst and SQL generator for the
AdventureWorks database.\n\nYour task is to translate a user's natural language
manufacturing or business question into a safe, correct SQL Server query and a
brief explanation.\n\nRules:\n1. Only generate read-only SQL.\n2. Allowed query
types are:\n    - SELECT\n    - Common Table Expressions (CTEs)\n    - ORDER BY\n
```

- GROUP BY\n - HAVING\n - JOINS\n - window functions\n - recursive CTEs if needed\n3. Do not generate:\n - INSERT\n - UPDATE\n - DELETE\n - DROP\n - ALTER\n - TRUNCATE\n - EXEC\n - MERGE\n - stored procedures\n4. The SQL dialect is Microsoft SQL Server (AdventureWorks).\n5. Prefer fully qualified table names, for example:\n Production.WorkOrder\n Production.Product\n Production.ProductInventory\n6. Use TOP for row limits, not LIMIT.\n7. If the question is ambiguous, make the smallest reasonable assumption and state it briefly.\n8. If the question cannot be answered from the provided schema, say so clearly.\n9. Use joins only where supported by the schema provided.\n10. Return output in valid JSON only.\n\nYou must return exactly this JSON structure:\n\n{\n "question_understanding": "short restatement of the task",\n "assumptions": ["assumption 1", "assumption 2"],\n "sql": "single SQL Server query as a string",\n "explanation": "brief explanation of how the query works",\n "result_shape": "what the output table should contain",\n "validation_checks": ["check 1", "check 2"],\n "plot_suggestion": {\n "recommended": true,\n "plot_type": "bar|line|pie|none",\n "x": "column name or null",\n "y": "column name or null",\n "title": "plot title or null"\n }\n}\n\nAdditional requirements:\n- The SQL must be executable as written.\n- Do not wrap SQL in markdown fences.\n- If no chart is appropriate, set plot_suggestion.plot_type to "none" and other plot fields to null.\n- If aggregation is used, ensure all non-aggregated selected columns are included in GROUP BY.\n- If ranking is requested, prefer ORDER BY and TOP, or window functions where appropriate.\n- If multi-level bill-of-materials logic is requested, use a recursive CTE.'},\n {'role': 'user',\n 'content': 'Database: AdventureWorks\nTask: Convert the following natural language request into SQL Server SQL.\n\nSchema\nsummary:\nProduction.WorkOrder\n- WorkOrderID (PK)\n- ProductID (FK -> Production.Product.ProductID)\n- OrderQty\n- StockedQty\n- ScrappedQty\n- StartDate\n- EndDate\n- DueDate\n- ScrapReasonID\n\nProduction.Product\n- ProductID (PK)\n- Name\n- ProductNumber\n- MakeFlag\n- FinishedGoodsFlag\n- Color\n- SafetyStockLevel\n- ReorderPoint\n- StandardCost\n- ListPrice\n- ProductSubcategoryID\n\nProduction.ProductInventory\n- ProductID (FK -> Production.Product.ProductID)\n- LocationID (FK -> Production.Location.LocationID)\n- Shelf\n- Bin\n- Quantity\n\nProduction.Location\n- LocationID (PK)\n- Name\n- CostRate\n- Availability\n\nProduction.WorkOrderRouting\n- WorkOrderID (FK -> Production.WorkOrder.WorkOrderID)\n- ProductID\n- OperationSequence\n- LocationID (FK -> Production.Location.LocationID)\n- ScheduledStartDate\n- ScheduledEndDate\n- ActualStartDate\n- ActualEndDate\n- PlannedCost\n- ActualCost\n\nProduction.BillofMaterials\n- BillofMaterialsID (PK)\n- ProductAssemblyID\n- ComponentID\n- StartDate\n- EndDate\n- UnitMeasureCode\n- BOMLevel\n- PerAssemblyQty\n\nProduction.TransactionHistory\n- TransactionID\n- ProductID\n- ReferenceOrderID\n- ReferenceOrderLineID\n- TransactionDate\n- TransactionType\n- Quantity\n- ActualCost\n\nBusiness context:\n- This database represents a manufacturing company.\n\nImportant table hints:\n- Work orders: Production.WorkOrder + Production.Product\n- Inventory:

```
Production.ProductInventory + Production.Location +
Production.Product\n\nNatural language request:\nWhich 10 products have the
highest on-hand quantity in total, and where is that stock located?\n\nOutput
requirements:\n- Return valid JSON only.\n']}]
```

```
[11]: response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=message # correct
)
print(response.choices[0].message.content)

{
  "question_understanding": "Identify the top 10 products with the highest on-
hand quantity and their stock locations.",
  "assumptions": [],
  "sql": "SELECT TOP 10 p.Name AS ProductName, SUM(pi.Quantity) AS TotalOnHand,
l.Name AS LocationName " +
    "FROM Production.Product AS p " +
    "JOIN Production.ProductInventory AS pi ON p.ProductID = pi.ProductID "
+
    "JOIN Production.Location AS l ON pi.LocationID = l.LocationID " +
    "GROUP BY p.Name, l.Name " +
    "ORDER BY TotalOnHand DESC;",
  "explanation": "This query selects the top 10 products based on the total on-
hand quantity from the ProductInventory table, joining with the Product and
Location tables to get product names and their respective stock locations. It
sums the quantities grouped by product and location, ordering the results in
descending order of total on-hand quantity.",
  "result_shape": "The output will have three columns: ProductName, TotalOnHand,
and LocationName.",
  "validation_checks": ["TotalOnHand should account for all locations for each
product", "Only products with available inventory will be counted"],
  "plot_suggestion": {
    "recommended": true,
    "plot_type": "bar",
    "x": "ProductName",
    "y": "TotalOnHand",
    "title": "Top 10 Products by On-Hand Quantity"
  }
}
```

```
[12]: query = """
WITH ProductOnHand AS (
  SELECT
    p.ProductID,
    p.Name,
    SUM(pi.Quantity) AS TotalQuantity
  FROM
```

```

        Production.Product AS p
    JOIN
        Production.ProductInventory AS pi
    ON p.ProductID = pi.ProductID
    GROUP BY
        p.ProductID, p.Name
),
RankedProducts AS (
    SELECT
        poh.ProductID,
        poh.Name,
        poh.TotalQuantity,
        ROW_NUMBER() OVER (ORDER BY poh.TotalQuantity DESC) AS rn
    FROM
        ProductOnHand AS poh
)
SELECT
    rp.ProductID,
    rp.Name,
    rp.TotalQuantity,
    pl.Name AS LocationName
FROM
    RankedProducts AS rp
JOIN
    Production.ProductInventory AS pi
ON rp.ProductID = pi.ProductID
JOIN
    Production.Location AS pl
ON pi.LocationID = pl.LocationID
WHERE
    rp.rn <= 10
ORDER BY
    rp.TotalQuantity DESC;
"""
table = pd.read_sql(query, conn)
table

```

C:\Users\emper\AppData\Local\Temp\ipykernel_55500\1083011272.py:42: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
table = pd.read_sql(query, conn)
```

```
[12]:
```

	ProductID	Name	TotalQuantity	LocationName
0	379	Hex Nut 7	1911	Tool Crib
1	379	Hex Nut 7	1911	Miscellaneous Storage
2	379	Hex Nut 7	1911	Subassembly

3	383	Hex Nut 23	1901	Tool Crib
4	383	Hex Nut 23	1901	Miscellaneous Storage
5	383	Hex Nut 23	1901	Subassembly
6	367	Thin-Jam Hex Nut 3	1901	Tool Crib
7	367	Thin-Jam Hex Nut 3	1901	Miscellaneous Storage
8	367	Thin-Jam Hex Nut 3	1901	Subassembly
9	387	Hex Nut 10	1888	Tool Crib
10	387	Hex Nut 10	1888	Miscellaneous Storage
11	387	Hex Nut 10	1888	Subassembly
12	393	Hex Nut 14	1880	Tool Crib
13	393	Hex Nut 14	1880	Miscellaneous Storage
14	393	Hex Nut 14	1880	Subassembly
15	489	Metal Tread Plate	1837	Tool Crib
16	489	Metal Tread Plate	1837	Miscellaneous Storage
17	489	Metal Tread Plate	1837	Subassembly
18	396	Hex Nut 18	1824	Tool Crib
19	396	Hex Nut 18	1824	Miscellaneous Storage
20	396	Hex Nut 18	1824	Subassembly
21	389	Hex Nut 2	1808	Tool Crib
22	389	Hex Nut 2	1808	Miscellaneous Storage
23	389	Hex Nut 2	1808	Subassembly
24	376	Hex Nut 6	1781	Tool Crib
25	376	Hex Nut 6	1781	Miscellaneous Storage
26	376	Hex Nut 6	1781	Subassembly
27	371	Thin-Jam Hex Nut 7	1781	Tool Crib
28	371	Thin-Jam Hex Nut 7	1781	Miscellaneous Storage
29	371	Thin-Jam Hex Nut 7	1781	Subassembly

[]:

1 Output Evaluation

1.0.1 Strenght

- Inventory aggregation at product level: **SUM(pi.Quantity) AS TotalQuantity**
- Products were ranked as expected: **ROW_NUMBER() OVER (ORDER BY poh.TotalQuantity DESC)**
- Filters the top 10 before joining them with the location table ### Weakness
- The output doesn't show the amount of stock at each location. The amount of stock in each location may also be important to a line manager. Therefore we should add: **pi.Quantity AS LocationQuantity**
- The validation check 2 "Make sure only the top 10 records based on total quantity are returned." needs to be redefined because there are more than 10 product rows due to the the fact that some products are spread across different locations. To fix this, we can include the following in the validation checks: "Verify that exactly 10 distinct ProductID values are returned.", "Verify that TotalQuantity equals the sum of LocationQuantity values for each product."

- “x”: “Name”. This appears too generic. The rp.Name AS ProductName and “x”: “Product-Name” should be used to correct this.

The highlighted quality-control issues are fixed using the **few-shot prompting techniques** as shown below

```
[14]: user_content2 = f"""
Database: AdventureWorks

Schema summary:
{database_schema}

Example:

Input:
Which 10 products have the highest on-hand quantity in total, and where is that
↪stock located?

Output:
{{
  "question_understanding": "Identify the top 10 products with the highest
↪total on-hand quantity and show the locations where that stock is held.",
  "assumptions": [
    "On-hand quantity means the sum of ProductInventory.Quantity across all
↪inventory locations for each product."
  ],
  "sql": "WITH ProductOnHand AS ( SELECT p.ProductID, p.Name AS ProductName,
↪SUM(pi.Quantity) AS TotalQuantity FROM Production.Product AS p JOIN
↪Production.ProductInventory AS pi ON p.ProductID = pi.ProductID GROUP BY p.
↪ProductID, p.Name ), RankedProducts AS ( SELECT poh.ProductID, poh.
↪ProductName, poh.TotalQuantity, ROW_NUMBER() OVER (ORDER BY poh.
↪TotalQuantity DESC) AS rn FROM ProductOnHand AS poh ) SELECT rp.ProductID,
↪rp.ProductName, rp.TotalQuantity, pl.Name AS LocationName, pi.Quantity AS
↪LocationQuantity FROM RankedProducts AS rp JOIN Production.ProductInventory
↪AS pi ON rp.ProductID = pi.ProductID JOIN Production.Location AS pl ON pi.
↪LocationID = pl.LocationID WHERE rp.rn <= 10 ORDER BY rp.TotalQuantity DESC,
↪rp.ProductName, pl.Name;",
  "explanation": "The query first aggregates on-hand quantity across all
↪locations for each product, then ranks products by total quantity and
↪selects the top 10. It finally joins back to inventory and location tables
↪to show where the stock is held and the quantity at each location.",
  "result_shape": "A table with ProductID, ProductName, TotalQuantity,
↪LocationName, and LocationQuantity for the top 10 products by total on-hand
↪inventory.",
  "validation_checks": [
    "Verify that exactly 10 distinct ProductID values are returned.",
  ]
}}
```

```

    "Verify that TotalQuantity equals the sum of LocationQuantity values for_
each product."
],
"plot_suggestion": {
    "recommended": true,
    "plot_type": "bar",
    "x": "ProductName",
    "y": "TotalQuantity",
    "title": "Top 10 Products by Total On-Hand Quantity"
}
}
}

---

Now answer:

{user_question}

Return valid JSON only.
"""

```

```

[15]: message = [
    {
        "role": "system",
        "content": system
    },
    {
        "role": "user",
        "content": user_content2    # MUST be a string
    }
]
message

```

```

[15]: [{'role': 'system',
        'content': 'You are an expert data analyst and SQL generator for the
AdventureWorks database.\n\nYour task is to translate a user's natural language
manufacturing or business question into a safe, correct SQL Server query and a
brief explanation.\n\nRules:\n1. Only generate read-only SQL.\n2. Allowed query
types are:\n    - SELECT\n    - Common Table Expressions (CTEs)\n    - ORDER BY\n    - GROUP BY\n    - HAVING\n    - JOINS\n    - window functions\n    - recursive CTEs
if needed\n3. Do not generate:\n    - INSERT\n    - UPDATE\n    - DELETE\n    -
DROP\n    - ALTER\n    - TRUNCATE\n    - EXEC\n    - MERGE\n    - stored
procedures\n4. The SQL dialect is Microsoft SQL Server (AdventureWorks).\n5.
Prefer fully qualified table names, for example:\n    Production.WorkOrder\n
Production.Product\n    Production.ProductInventory\n6. Use TOP for row limits,
not LIMIT.\n7. If the question is ambiguous, make the smallest reasonable
assumption and state it briefly.\n8. If the question cannot be answered from the

```

provided schema, say so clearly.\n9. Use joins only where supported by the schema provided.\n10. Return output in valid JSON only.\n\nYou must return exactly this JSON structure:\n\n{\n "question_understanding": "short restatement of the task",\n "assumptions": ["assumption 1", "assumption 2"],\n "sql": "single SQL Server query as a string",\n "explanation": "brief explanation of how the query works",\n "result_shape": "what the output table should contain",\n "validation_checks": ["check 1", "check 2"],\n "plot_suggestion": {\n "recommended": true,\n "plot_type": "bar|line|pie|none",\n "x": "column name or null",\n "y": "column name or null",\n "title": "plot title or null"\n }\n}\n\nAdditional requirements:\n- The SQL must be executable as written.\n- Do not wrap SQL in markdown fences.\n- If no chart is appropriate, set plot_suggestion.plot_type to "none" and other plot fields to null.\n- If aggregation is used, ensure all non-aggregated selected columns are included in GROUP BY.\n- If ranking is requested, prefer ORDER BY and TOP, or window functions where appropriate.\n- If multi-level bill-of-materials logic is requested, use a recursive CTE.'},\n {'role': 'user',\n 'content': '\nDatabase: AdventureWorks\n\nSchema summary:\nProduction.WorkOrder\n- WorkOrderID (PK)\n- ProductID (FK -> Production.Product.ProductID)\n- OrderQty\n- StockedQty\n- ScrappedQty\n- StartDate\n- EndDate\n- DueDate\n- ScrapReasonID\n\nProduction.Product\n- ProductID (PK)\n- Name\n- ProductNumber\n- MakeFlag\n- FinishedGoodsFlag\n- Color\n- SafetyStockLevel\n- ReorderPoint\n- StandardCost\n- ListPrice\n- ProductSubcategoryID\n\nProduction.ProductInventory\n- ProductID (FK -> Production.Product.ProductID)\n- LocationID (FK -> Production.Location.LocationID)\n- Shelf\n- Bin\n- Quantity\n\nProduction.Location\n- LocationID (PK)\n- Name\n- CostRate\n- Availability\n\nProduction.WorkOrderRouting\n- WorkOrderID (FK -> Production.WorkOrder.WorkOrderID)\n- ProductID\n- OperationSequence\n- LocationID (FK -> Production.Location.LocationID)\n- ScheduledStartDate\n- ScheduledEndDate\n- ActualStartDate\n- ActualEndDate\n- PlannedCost\n- ActualCost\n\nProduction.BillOfMaterials\n- BillOfMaterialsID (PK)\n- ProductAssemblyID\n- ComponentID\n- StartDate\n- EndDate\n- UnitMeasureCode\n- BOMLevel\n- PerAssemblyQty\n\nProduction.TransactionHistory\n- TransactionID\n- ProductID\n- ReferenceOrderID\n- ReferenceOrderLineID\n- TransactionDate\n- TransactionType\n- Quantity\n- ActualCost\n\nExample:\n\nInput:\nWhich 10 products have the highest on-hand quantity in total, and where is that stock located?\n\nOutput:\n{\n "question_understanding": "Identify the top 10 products with the highest total on-hand quantity and show the locations where that stock is held.",\n "assumptions": [\n "On-hand quantity means the sum of ProductInventory.Quantity across all inventory locations for each product."\n],\n "sql": "WITH ProductOnHand AS (SELECT p.ProductID, p.Name AS ProductName, SUM(pi.Quantity) AS TotalQuantity FROM Production.Product AS p JOIN Production.ProductInventory AS pi ON p.ProductID = pi.ProductID GROUP BY p.ProductID, p.Name), RankedProducts AS (SELECT poh.ProductID, poh.ProductName, poh.TotalQuantity, ROW_NUMBER() OVER (ORDER BY poh.TotalQuantity DESC) AS rn FROM ProductOnHand AS poh) SELECT rp.ProductID,

```
rp.ProductName, rp.TotalQuantity, pl.Name AS LocationName, pi.Quantity AS
LocationQuantity FROM RankedProducts AS rp JOIN Production.ProductInventory AS
pi ON rp.ProductID = pi.ProductID JOIN Production.Location AS pl ON
pi.LocationID = pl.LocationID WHERE rp.rn <= 10 ORDER BY rp.TotalQuantity DESC,
rp.ProductName, pl.Name;","\n "explanation": "The query first aggregates on-hand
quantity across all locations for each product, then ranks products by total
quantity and selects the top 10. It finally joins back to inventory and location
tables to show where the stock is held and the quantity at each location.",\n
"result_shape": "A table with ProductID, ProductName, TotalQuantity,
LocationName, and LocationQuantity for the top 10 products by total on-hand
inventory.",\n "validation_checks": [\n    "Verify that exactly 10 distinct
ProductID values are returned.",\n    "Verify that TotalQuantity equals the sum
of LocationQuantity values for each product." ],\n "plot_suggestion": {\n
"recommended": true,\n    "plot_type": "bar",\n    "x": "ProductName",\n    "y":
"TotalQuantity",\n    "title": "Top 10 Products by Total On-Hand Quantity"\n
}\n}\n\n---\n\nNow answer:\n\nWhich 10 products have the highest on-hand
quantity in total, and where is that stock located?\n\nReturn valid JSON
only.\n'}}
```

```
[16]: response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=message # correct
)
print(response.choices[0].message.content)
```

```
{
  "question_understanding": "Identify the top 10 products with the highest total
on-hand quantity and show the locations where that stock is held.",
  "assumptions": [
    "On-hand quantity means the sum of ProductInventory.Quantity across all
inventory locations for each product."
  ],
  "sql": "WITH ProductOnHand AS ( SELECT p.ProductID, p.Name AS ProductName,
SUM(pi.Quantity) AS TotalQuantity FROM Production.Product AS p JOIN
Production.ProductInventory AS pi ON p.ProductID = pi.ProductID GROUP BY
p.ProductID, p.Name ), RankedProducts AS ( SELECT poh.ProductID,
poh.ProductName, poh.TotalQuantity, ROW_NUMBER() OVER (ORDER BY
poh.TotalQuantity DESC) AS rn FROM ProductOnHand AS poh ) SELECT rp.ProductID,
rp.ProductName, rp.TotalQuantity, pl.Name AS LocationName, pi.Quantity AS
LocationQuantity FROM RankedProducts AS rp JOIN Production.ProductInventory AS
pi ON rp.ProductID = pi.ProductID JOIN Production.Location AS pl ON
pi.LocationID = pl.LocationID WHERE rp.rn <= 10 ORDER BY rp.TotalQuantity DESC,
rp.ProductName, pl.Name;",
  "explanation": "The query first aggregates on-hand quantity across all
locations for each product, then ranks products by total quantity and selects
the top 10. It finally joins back to inventory and location tables to show where
the stock is held and the quantity at each location.",
  "result_shape": "A table with ProductID, ProductName, TotalQuantity,
```

```

LocationName, and LocationQuantity for the top 10 products by total on-hand
inventory.",
  "validation_checks": [
    "Verify that exactly 10 distinct ProductID values are returned.",
    "Verify that TotalQuantity equals the sum of LocationQuantity values for
each product."
  ],
  "plot_suggestion": {
    "recommended": true,
    "plot_type": "bar",
    "x": "ProductName",
    "y": "TotalQuantity",
    "title": "Top 10 Products by Total On-Hand Quantity"
  }
}

```

```

[17]: query2 = """
WITH ProductOnHand AS (
  SELECT
    p.ProductID,
    p.Name AS ProductName,
    SUM(pi.Quantity) AS TotalQuantity
  FROM
    Production.Product AS p
  JOIN
    Production.ProductInventory AS pi
    ON p.ProductID = pi.ProductID
  GROUP BY
    p.ProductID, p.Name
),
RankedProducts AS (
  SELECT
    poh.ProductID,
    poh.ProductName,
    poh.TotalQuantity,
    ROW_NUMBER() OVER (ORDER BY poh.TotalQuantity DESC) AS rn
  FROM
    ProductOnHand AS poh
)
SELECT
  rp.ProductID,
  rp.ProductName,
  rp.TotalQuantity,
  pl.Name AS LocationName,
  pi.Quantity AS LocationQuantity
FROM
  RankedProducts AS rp

```

```

JOIN
    Production.ProductInventory AS pi
    ON rp.ProductID = pi.ProductID
JOIN
    Production.Location AS pl
    ON pi.LocationID = pl.LocationID
WHERE
    rp.rn <= 10
ORDER BY
    rp.TotalQuantity DESC,
    rp.ProductName,
    pl.Name;
"""

table2 = pd.read_sql(query2, conn)
table2

```

C:\Users\emper\AppData\Local\Temp\ipykernel_55500\1676636856.py:46: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
table2 = pd.read_sql(query2, conn)
```

```
[17]:
```

	ProductID	ProductName	TotalQuantity	LocationName \
0	379	Hex Nut 7	1911	Miscellaneous Storage
1	379	Hex Nut 7	1911	Subassembly
2	379	Hex Nut 7	1911	Tool Crib
3	383	Hex Nut 23	1901	Miscellaneous Storage
4	383	Hex Nut 23	1901	Subassembly
5	383	Hex Nut 23	1901	Tool Crib
6	367	Thin-Jam Hex Nut 3	1901	Miscellaneous Storage
7	367	Thin-Jam Hex Nut 3	1901	Subassembly
8	367	Thin-Jam Hex Nut 3	1901	Tool Crib
9	387	Hex Nut 10	1888	Miscellaneous Storage
10	387	Hex Nut 10	1888	Subassembly
11	387	Hex Nut 10	1888	Tool Crib
12	393	Hex Nut 14	1880	Miscellaneous Storage
13	393	Hex Nut 14	1880	Subassembly
14	393	Hex Nut 14	1880	Tool Crib
15	489	Metal Tread Plate	1837	Miscellaneous Storage
16	489	Metal Tread Plate	1837	Subassembly
17	489	Metal Tread Plate	1837	Tool Crib
18	396	Hex Nut 18	1824	Miscellaneous Storage
19	396	Hex Nut 18	1824	Subassembly
20	396	Hex Nut 18	1824	Tool Crib
21	389	Hex Nut 2	1808	Miscellaneous Storage
22	389	Hex Nut 2	1808	Subassembly

23	389	Hex Nut 2	1808	Tool Crib
24	376	Hex Nut 6	1781	Miscellaneous Storage
25	376	Hex Nut 6	1781	Subassembly
26	376	Hex Nut 6	1781	Tool Crib
27	371	Thin-Jam Hex Nut 7	1781	Miscellaneous Storage
28	371	Thin-Jam Hex Nut 7	1781	Subassembly
29	371	Thin-Jam Hex Nut 7	1781	Tool Crib

	LocationQuantity
0	897
1	691
2	323
3	676
4	638
5	587
6	689
7	569
8	643
9	710
10	654
11	524
12	780
13	497
14	603
15	619
16	606
17	612
18	585
19	603
20	636
21	691
22	460
23	657
24	649
25	547
26	585
27	649
28	547
29	585

```
[18]: #table2.to_csv("hoq_stock_location_df.csv", index=False)
```

```
[220]: top_10 = pd.read_csv("hoq_stock_location_df.csv")
top_10.head()
```

```
[220]:   ProductID ProductName TotalQuantity LocationName \
0        379   Hex Nut 7          1911 Miscellaneous Storage
```


1	379	Hex Nut 7	1911	Subassembly
2	379	Hex Nut 7	1911	Tool Crib
3	383	Hex Nut 23	1901	Miscellaneous Storage
4	383	Hex Nut 23	1901	Subassembly

	LocationQuantity
0	897
1	691
2	323
3	676
4	638

```
[230]: product_col = "ProductName"
location_col = "LocationName"
qty_col      = "LocationQuantity"

# -----
# Create pivot table for stacked chart
# -----
pivot_df = top_10.pivot_table(
    index=product_col,
    columns=location_col,
    values=qty_col,
    aggfunc="sum",
    fill_value=0
)

# Optional: sort by total quantity descending
pivot_df["Total"] = pivot_df.sum(axis=1)
pivot_df = pivot_df.sort_values("Total", ascending=True)
pivot_df = pivot_df.drop(columns="Total")

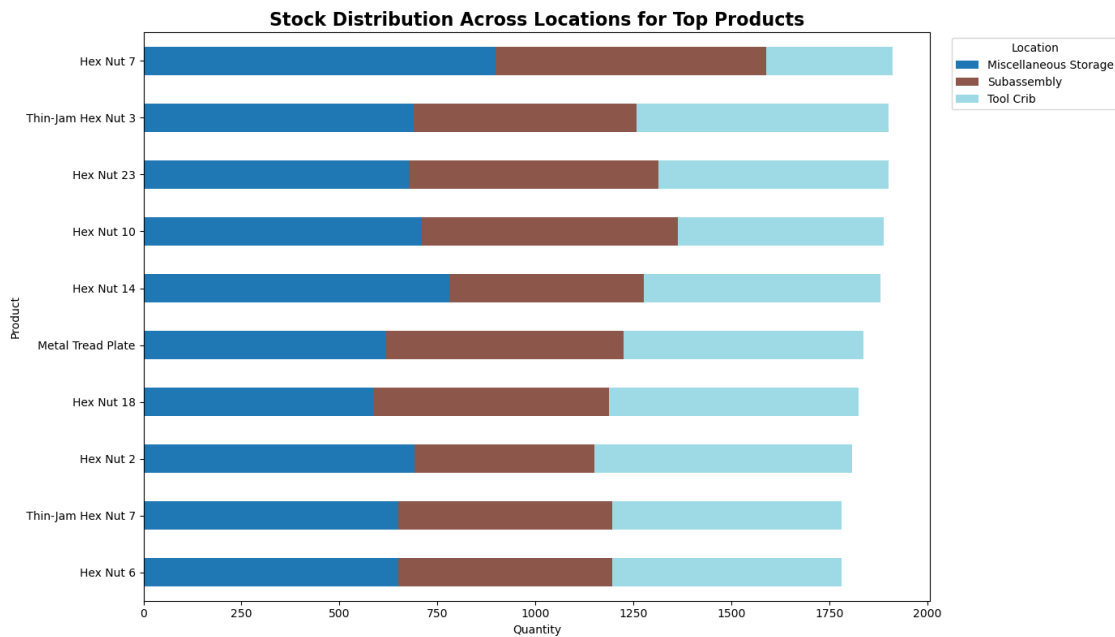
# -----
# Plot horizontal stacked bar chart
# -----
plt.figure(figsize=(14,8))

pivot_df.plot(
    kind="barh",
    stacked=True,
    figsize=(14,8),
    colormap="tab20"
)

plt.title("Stock Distribution Across Locations for Top Products", fontsize=16,
    ↪weight="bold")
plt.xlabel("Quantity")
```

```
plt.ylabel("Product")
plt.legend(title="Location", bbox_to_anchor=(1.02,1), loc="upper left")
plt.tight_layout()
plt.show()
```

<Figure size 1400x800 with 0 Axes>



The output of table two confirmed that all the issues highlighted in table1 has been resolved.

```
[20]: #plt.bar(x=table2.ProductName, y= table2.TotalQuantity)
      #plt.title("Top 10 Products by Total On-Hand Quantity")
      #plt.show()
```

```
[21]: user_question2 = "By manufacturing location (work center), count the number of_
      ↪operations scheduled per week."
```

```
[22]: user_content3 = f"""Database: AdventureWorks
Task: Convert the following natural language request into SQL Server SQL.

Schema summary:
{database_schema}

Business context:
- This database represents a manufacturing company.

Important table hints:
- Work orders: Production.WorkOrder + Production.Product
```

```
- Inventory: Production.ProductInventory + Production.Location + Production.
  ↳Product
- Operation Schedule: Production.WorkOrderRouting + Production.Location
```

Natural language request:
{user_question2}

Output requirements:
- Return valid JSON only.
""

```
[23]: message2 = [
    {
        "role": "system",
        "content": system
    },
    {
        "role": "user",
        "content": user_content3    # MUST be a string
    }
]
message2
```

```
[23]: [{ 'role': 'system',
    'content': 'You are an expert data analyst and SQL generator for the
AdventureWorks database.\n\nYour task is to translate a user's natural language
manufacturing or business question into a safe, correct SQL Server query and a
brief explanation.\n\nRules:\n1. Only generate read-only SQL.\n2. Allowed query
types are:\n    - SELECT\n    - Common Table Expressions (CTEs)\n    - ORDER BY\n    - GROUP BY\n    - HAVING\n    - JOINS\n    - window functions\n    - recursive CTEs
if needed\n3. Do not generate:\n    - INSERT\n    - UPDATE\n    - DELETE\n    -
DROP\n    - ALTER\n    - TRUNCATE\n    - EXEC\n    - MERGE\n    - stored
procedures\n4. The SQL dialect is Microsoft SQL Server (AdventureWorks).\n5.
Prefer fully qualified table names, for example:\n    Production.WorkOrder\n
Production.Product\n    Production.ProductInventory\n6. Use TOP for row limits,
not LIMIT.\n7. If the question is ambiguous, make the smallest reasonable
assumption and state it briefly.\n8. If the question cannot be answered from the
provided schema, say so clearly.\n9. Use joins only where supported by the
schema provided.\n10. Return output in valid JSON only.\n\nYou must return
exactly this JSON structure:\n\n{\n    "question_understanding": "short
restatement of the task",\n    "assumptions": ["assumption 1", "assumption 2"],\n
"sql": "single SQL Server query as a string",\n    "explanation": "brief
explanation of how the query works",\n    "result_shape": "what the output table
should contain",\n    "validation_checks": ["check 1", "check 2"],\n
"plot_suggestion": {\n        "recommended": true,\n        "plot_type":
"bar|line|pie|none",\n        "x": "column name or null",\n        "y": "column name or
null",\n        "title": "plot title or null"\n    }\n}\n\nAdditional requirements:\n-
```

The SQL must be executable as written.\n- Do not wrap SQL in markdown fences.\n- If no chart is appropriate, set plot_suggestion.plot_type to "none" and other plot fields to null.\n- If aggregation is used, ensure all non-aggregated selected columns are included in GROUP BY.\n- If ranking is requested, prefer ORDER BY and TOP, or window functions where appropriate.\n- If multi-level bill-of-materials logic is requested, use a recursive CTE.'},

```
{'role': 'user',
  'content': 'Database: AdventureWorks\nTask: Convert the following natural
language request into SQL Server SQL.\n\nSchema
summary:\nProduction.WorkOrder\n- WorkOrderID (PK)\n- ProductID (FK ->
Production.Product.ProductID)\n- OrderQty\n- StockedQty\n- ScrappedQty\n-
StartDate\n- EndDate\n- DueDate\n- ScrapReasonID\n\nProduction.Product\n-
ProductID (PK)\n- Name\n- ProductNumber\n- MakeFlag\n- FinishedGoodsFlag\n-
Color\n- SafetyStockLevel\n- ReorderPoint\n- StandardCost\n- ListPrice\n-
ProductSubcategoryID\n\nProduction.ProductInventory\n- ProductID (FK ->
Production.Product.ProductID)\n- LocationID (FK ->
Production.Location.LocationID)\n- Shelf\n- Bin\n-
Quantity\n\nProduction.Location\n- LocationID (PK)\n- Name\n- CostRate\n-
Availability\n\nProduction.WorkOrderRouting\n- WorkOrderID (FK ->
Production.WorkOrder.WorkOrderID)\n- ProductID\n- OperationSequence\n-
LocationID (FK -> Production.Location.LocationID)\n- ScheduledStartDate\n-
ScheduledEndDate\n- ActualStartDate\n- ActualEndDate\n- PlannedCost\n-
ActualCost\n\nProduction.BillOfMaterials\n- BillOfMaterialsID (PK)\n-
ProductAssemblyID\n- ComponentID\n- StartDate\n- EndDate\n- UnitMeasureCode\n-
BOMLevel\n- PerAssemblyQty\n\nProduction.TransactionHistory\n- TransactionID\n-
ProductID\n- ReferenceOrderID\n- ReferenceOrderLineID\n- TransactionDate\n-
TransactionType\n- Quantity\n- ActualCost\n\nBusiness context:\n- This database
represents a manufacturing company.\n\nImportant table hints:\n- Work orders:
Production.WorkOrder + Production.Product\n- Inventory:
Production.ProductInventory + Production.Location + Production.Product\n-
Operation Schedule: Production.WorkOrderRouting + Production.Location\n\nNatural
language request:\nBy manufacturing location (work center), count the number of
operations scheduled per week.\n\nOutput requirements:\n- Return valid JSON
only.\n'}}
```

```
[24]: response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=message2 # correct
)
print(response.choices[0].message.content)
```

```
{
  "question_understanding": "Count the number of scheduled operations by
manufacturing location per week.",
  "assumptions": [
    "Each operation is characterized by a unique work order routing entry.",
    "The week starts on Monday."
  ],
}
```

```
"sql": "WITH WeeklyOperations AS (\n    SELECT \n        pl.Name AS\n        LocationName,\n        DATEPART(YEAR, wr.ScheduledStartDate) AS Year,\n        DATEPART(WEEK, wr.ScheduledStartDate) AS Week,\n        COUNT(*) AS\n        OperationCount\n    FROM \n        Production.WorkOrderRouting wr\n    JOIN \n        Production.Location pl ON wr.LocationID = pl.LocationID\n    GROUP BY \n        pl.Name, DATEPART(YEAR, wr.ScheduledStartDate), DATEPART(WEEK,\n        wr.ScheduledStartDate)\n)\nSELECT \n    LocationName,\n    Year,\n    Week,\n    OperationCount\nFROM \n    WeeklyOperations\nORDER BY \n    LocationName, Year,\n    Week;"
```

"explanation": "The query uses a Common Table Expression (CTE) to first aggregate the number of operations scheduled for each location per week, based on the 'ScheduledStartDate' from the 'Production.WorkOrderRouting' table. It groups the results by location, year, and week number, and counts the number of operations for those groups. Finally, it selects these results, ordered by location, year, and week."

"result_shape": "The output table will contain LocationName, Year, Week, and OperationCount columns."

"validation_checks": [
 "Check that all operations are counted accurately per location and week.",
 "Check that the week starts on Monday as per the SQL Server default setting."

```
],  

"plot_suggestion": {  

    "recommended": true,  

    "plot_type": "bar",  

    "x": "LocationName",  

    "y": "OperationCount",  

    "title": "Weekly Scheduled Operations by Manufacturing Location"  

}
```

```
}
```

```
[25]: query3 = """  

SELECT  

    DATEADD(WEEK, DATEDIFF(WEEK, 0, wor.ScheduledStartDate), 0) AS WeekStart,  

    l.Name AS LocationName,  

    COUNT(*) AS OperationCount  

FROM Production.WorkOrderRouting AS wor  

JOIN Production.Location AS l  

    ON wor.LocationID = l.LocationID  

WHERE wor.ScheduledStartDate IS NOT NULL  

GROUP BY  

    DATEADD(WEEK, DATEDIFF(WEEK, 0, wor.ScheduledStartDate), 0),  

    l.Name  

ORDER BY  

    WeekStart,  

    LocationName;  

"""
```

```
table3 = pd.read_sql(query3, conn)
table3
```

C:\Users\emper\AppData\Local\Temp\ipykernel_55500\3770967894.py:17: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
table3 = pd.read_sql(query3, conn)
```

```
[25]:
```

	WeekStart	LocationName	OperationCount
0	2011-05-30	Debur and Polish	5
1	2011-05-30	Final Assembly	39
2	2011-05-30	Frame Forming	17
3	2011-05-30	Frame Welding	11
4	2011-05-30	Paint	2
...	...	...	...
1064	2014-06-02	Frame Forming	25
1065	2014-06-02	Frame Welding	17
1066	2014-06-02	Paint	2
1067	2014-06-02	Specialized Paint	2
1068	2014-06-02	Subassembly	82

```
[1069 rows x 3 columns]
```

2 Output Evaluation

2.1 Strength

- Correct table selection
- Consistent JSON output
- Correct aggregation/grouping logic
- Correct time grouping ## Weakness
- Failure to exclude null schedule dates which may cause logically invalid grouping. We need to add **WHERE WR.ScheduledStartDate IS NOT NULL** to correct this
- Change the ordering from Location, year week to year, week, location to ensure ease of trend analysis
- Use both week and year for plotting instead of week alone. This will prevent misleading axis labelling over multiple years

```
[27]: user_content4 = f"""
Database: AdventureWorks
```

```
Schema summary:
{database_schema}
```

```
Example:
```

```
Input:
```

```

By manufacturing location (work center), count the number of operations
↳scheduled per week.
Output:
{{
  "question_understanding": "Count the number of routing operations scheduled
↳each week for each manufacturing location (work center).",
  "assumptions": [
    "Each row in Production.WorkOrderRouting represents one scheduled operation.
↳",
    "Weekly grouping is based on ScheduledStartDate.",
    "Manufacturing locations are represented by Production.Location."
  ],
  "sql": "SELECT L.Name AS LocationName, DATEADD(WEEK, DATEDIFF(WEEK, 0, WR.
↳ScheduledStartDate), 0) AS WeekStart, COUNT(*) AS OperationCount FROM
↳Production.WorkOrderRouting WR JOIN Production.Location L ON WR.LocationID =
↳L.LocationID WHERE WR.ScheduledStartDate IS NOT NULL GROUP BY L.Name,
↳DATEADD(WEEK, DATEDIFF(WEEK, 0, WR.ScheduledStartDate), 0) ORDER BY
↳WeekStart, LocationName;",
  "explanation": "The query joins routing rows to manufacturing locations,
↳groups them by the start of the scheduled week, and counts the number of
↳operations scheduled for each work center in each week.",
  "result_shape": "A table containing LocationName, WeekStart, and
↳OperationCount.",
  "validation_checks": [
    "Verify that rows with null ScheduledStartDate are excluded.",
    "Verify that each WorkOrderRouting row is counted once.",
    "Check that OperationCount is grouped by both location and scheduled week."
  ],
  "plot_suggestion": {{
    "recommended": true,
    "plot_type": "line",
    "x": "WeekStart",
    "y": "OperationCount",
    "title": "Weekly Scheduled Operations by Manufacturing Location"
  }}
}}

```

Now answer:

```
{user_question2}
```

Return valid JSON only.

```
"""
```

```
[28]: message2 = [
    {
        "role": "system",
        "content": system
    },
    {
        "role": "user",
        "content": user_content4    # MUST be a string
    }
]
message2
```

```
[28]: [{ 'role': 'system',
        'content': 'You are an expert data analyst and SQL generator for the
AdventureWorks database.\n\nYour task is to translate a user's natural language
manufacturing or business question into a safe, correct SQL Server query and a
brief explanation.\n\nRules:\n1. Only generate read-only SQL.\n2. Allowed query
types are:\n    - SELECT\n    - Common Table Expressions (CTEs)\n    - ORDER BY\n    - GROUP BY\n    - HAVING\n    - JOINS\n    - window functions\n    - recursive CTEs
if needed\n3. Do not generate:\n    - INSERT\n    - UPDATE\n    - DELETE\n    -
DROP\n    - ALTER\n    - TRUNCATE\n    - EXEC\n    - MERGE\n    - stored
procedures\n4. The SQL dialect is Microsoft SQL Server (AdventureWorks).\n5.
Prefer fully qualified table names, for example:\n    Production.WorkOrder\n
Production.Product\n    Production.ProductInventory\n6. Use TOP for row limits,
not LIMIT.\n7. If the question is ambiguous, make the smallest reasonable
assumption and state it briefly.\n8. If the question cannot be answered from the
provided schema, say so clearly.\n9. Use joins only where supported by the
schema provided.\n10. Return output in valid JSON only.\n\nYou must return
exactly this JSON structure:\n\n{\n  "question_understanding": "short
restatement of the task",\n  "assumptions": ["assumption 1", "assumption 2"],\n
"sql": "single SQL Server query as a string",\n  "explanation": "brief
explanation of how the query works",\n  "result_shape": "what the output table
should contain",\n  "validation_checks": ["check 1", "check 2"],\n
"plot_suggestion": {\n    "recommended": true,\n    "plot_type":
"bar|line|pie|none",\n    "x": "column name or null",\n    "y": "column name or
null",\n    "title": "plot title or null"\n  }\n}\n\nAdditional requirements:\n-
The SQL must be executable as written.\n- Do not wrap SQL in markdown fences.\n-
If no chart is appropriate, set plot_suggestion.plot_type to "none" and other
plot fields to null.\n- If aggregation is used, ensure all non-aggregated
selected columns are included in GROUP BY.\n- If ranking is requested, prefer
ORDER BY and TOP, or window functions where appropriate.\n- If multi-level bill-
of-materials logic is requested, use a recursive CTE.'},
  { 'role': 'user',
    'content': '\nDatabase: AdventureWorks\n\nSchema
summary:\nProduction.WorkOrder\n- WorkOrderID (PK)\n- ProductID (FK ->
Production.Product.ProductID)\n- OrderQty\n- StockedQty\n- ScrappedQty\n-
StartDate\n- EndDate\n- DueDate\n- ScrapReasonID\n\nProduction.Product\n-
```


ProductID (PK)\n- Name\n- ProductNumber\n- MakeFlag\n- FinishedGoodsFlag\n- Color\n- SafetyStockLevel\n- ReorderPoint\n- StandardCost\n- ListPrice\n- ProductSubcategoryID\n\nProduction.ProductInventory\n- ProductID (FK -> Production.Product.ProductID)\n- LocationID (FK -> Production.Location.LocationID)\n- Shelf\n- Bin\n- Quantity\n\nProduction.Location\n- LocationID (PK)\n- Name\n- CostRate\n- Availability\n\nProduction.WorkOrderRouting\n- WorkOrderID (FK -> Production.WorkOrder.WorkOrderID)\n- ProductID\n- OperationSequence\n- LocationID (FK -> Production.Location.LocationID)\n- ScheduledStartDate\n- ScheduledEndDate\n- ActualStartDate\n- ActualEndDate\n- PlannedCost\n- ActualCost\n\nProduction.BillOfMaterials\n- BillOfMaterialsID (PK)\n- ProductAssemblyID\n- ComponentID\n- StartDate\n- EndDate\n- UnitMeasureCode\n- BOMLevel\n- PerAssemblyQty\n\nProduction.TransactionHistory\n- TransactionID\n- ProductID\n- ReferenceOrderID\n- ReferenceOrderLineID\n- TransactionDate\n- TransactionType\n- Quantity\n- ActualCost\n\nExample:\n\nInput:\nBy manufacturing location (work center), count the number of operations scheduled per week.\n\nOutput:\n{\n "question_understanding": "Count the number of routing operations scheduled each week for each manufacturing location (work center).",\n "assumptions": [\n "Each row in Production.WorkOrderRouting represents one scheduled operation.",\n "Weekly grouping is based on ScheduledStartDate.",\n "Manufacturing locations are represented by Production.Location."],\n "sql": "SELECT L.Name AS LocationName, DATEADD(WEEK, DATEDIFF(WEEK, 0, WR.ScheduledStartDate), 0) AS WeekStart, COUNT(*) AS OperationCount FROM Production.WorkOrderRouting WR JOIN Production.Location L ON WR.LocationID = L.LocationID WHERE WR.ScheduledStartDate IS NOT NULL GROUP BY L.Name, DATEADD(WEEK, DATEDIFF(WEEK, 0, WR.ScheduledStartDate), 0) ORDER BY WeekStart, LocationName;",\n "explanation": "The query joins routing rows to manufacturing locations, groups them by the start of the scheduled week, and counts the number of operations scheduled for each work center in each week.",\n "result_shape": "A table containing LocationName, WeekStart, and OperationCount.",\n "validation_checks": [\n "Verify that rows with null ScheduledStartDate are excluded.",\n "Verify that each WorkOrderRouting row is counted once.",\n "Check that OperationCount is grouped by both location and scheduled week."],\n "plot_suggestion": {\n "recommended": true,\n "plot_type": "line",\n "x": "WeekStart",\n "y": "OperationCount",\n "title": "Weekly Scheduled Operations by Manufacturing Location" }\n}\n\n---\n\nNow answer:\n\nBy manufacturing location (work center), count the number of operations scheduled per week.\n\nReturn valid JSON only.\n']}]

```
[29]: response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=message2 # correct
    )
print(response.choices[0].message.content)
```

```
{
    "question_understanding": "Count the number of routing operations scheduled
```

```

each week for each manufacturing location (work center).",
  "assumptions": [
    "Each row in Production.WorkOrderRouting represents one scheduled operation.",
    "Weekly grouping is based on ScheduledStartDate.",
    "Manufacturing locations are represented by Production.Location."
  ],
  "sql": "SELECT L.Name AS LocationName, DATEADD(WEEK, DATEDIFF(WEEK, 0, WR.ScheduledStartDate), 0) AS WeekStart, COUNT(*) AS OperationCount FROM Production.WorkOrderRouting WR JOIN Production.Location L ON WR.LocationID = L.LocationID WHERE WR.ScheduledStartDate IS NOT NULL GROUP BY L.Name, DATEADD(WEEK, DATEDIFF(WEEK, 0, WR.ScheduledStartDate), 0) ORDER BY WeekStart, LocationName;",
  "explanation": "The query joins routing rows to manufacturing locations, groups them by the start of the scheduled week, and counts the number of operations scheduled for each work center in each week.",
  "result_shape": "A table containing LocationName, WeekStart, and OperationCount.",
  "validation_checks": [
    "Verify that rows with null ScheduledStartDate are excluded.",
    "Verify that each WorkOrderRouting row is counted once.",
    "Check that OperationCount is grouped by both location and scheduled week."
  ],
  "plot_suggestion": {
    "recommended": true,
    "plot_type": "line",
    "x": "WeekStart",
    "y": "OperationCount",
    "title": "Weekly Scheduled Operations by Manufacturing Location"
  }
}

```

```

[30]: query4 = """
SELECT
    l.Name AS LocationName,
    DATEADD(WEEK, DATEDIFF(WEEK, 0, wr.ScheduledStartDate), 0) AS WeekStart,
    COUNT(*) AS OperationCount
FROM Production.WorkOrderRouting AS wr
JOIN Production.Location AS l
    ON wr.LocationID = l.LocationID
WHERE wr.ScheduledStartDate IS NOT NULL
GROUP BY
    l.Name,
    DATEADD(WEEK, DATEDIFF(WEEK, 0, wr.ScheduledStartDate), 0)
ORDER BY
    WeekStart,
    LocationName;

```

```

"""
table4 = pd.read_sql(query4, conn)
table4

```

C:\Users\emper\AppData\Local\Temp\ipykernel_55500\3284692254.py:17: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
table4 = pd.read_sql(query4, conn)
```

```
[30]:
```

	LocationName	WeekStart	OperationCount
0	Debur and Polish	2011-05-30	5
1	Final Assembly	2011-05-30	39
2	Frame Forming	2011-05-30	17
3	Frame Welding	2011-05-30	11
4	Paint	2011-05-30	2
...	...	...	...
1064	Frame Forming	2014-06-02	25
1065	Frame Welding	2014-06-02	17
1066	Paint	2014-06-02	2
1067	Specialized Paint	2014-06-02	2
1068	Subassembly	2014-06-02	82

[1069 rows x 3 columns]

```
[31]: #table4.to_csv("op_schedule_per_week_count_df.csv", index=False)
```

```
[216]: operation_sched = pd.read_csv("op_schedule_per_week_count_df.csv")
operation_sched
```

```
[216]:
```

	LocationName	WeekStart	OperationCount
0	Debur and Polish	2011-05-30	5
1	Final Assembly	2011-05-30	39
2	Frame Forming	2011-05-30	17
3	Frame Welding	2011-05-30	11
4	Paint	2011-05-30	2
...	...	...	...
1064	Frame Forming	2014-06-02	25
1065	Frame Welding	2014-06-02	17
1066	Paint	2014-06-02	2
1067	Specialized Paint	2014-06-02	2
1068	Subassembly	2014-06-02	82

[1069 rows x 3 columns]

```
[233]: # Assign columns
date_col = "WeekStart"
location_col = "LocationName"
```

```

value_col = "OperationCount"

# Convert date column
operation_sched[date_col] = pd.to_datetime(operation_sched[date_col])

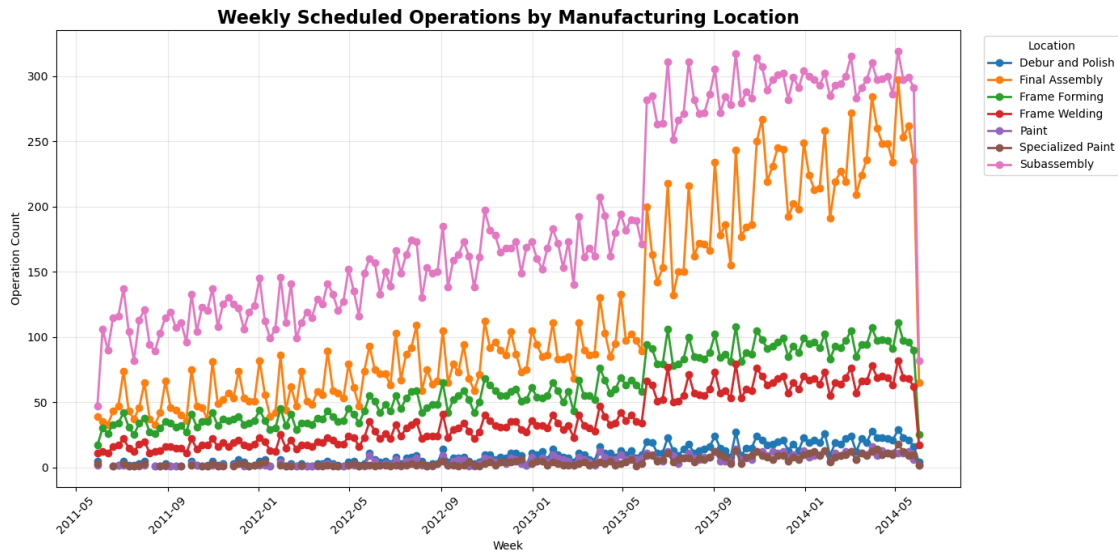
# -----
# Pivot for multi-line chart
# -----
pivot_df = operation_sched.pivot_table(
    index=date_col,
    columns=location_col,
    values=value_col,
    aggfunc="sum"
)

# -----
# Plot
# -----
plt.figure(figsize=(14,7))

for col in pivot_df.columns:
    plt.plot(
        pivot_df.index,
        pivot_df[col],
        marker="o",
        linewidth=2,
        label=col
    )

plt.title("Weekly Scheduled Operations by Manufacturing Location", fontsize=16,
        weight="bold")
plt.xlabel("Week")
plt.ylabel("Operation Count")
plt.legend(title="Location", bbox_to_anchor=(1.02,1), loc="upper left")
plt.xticks(rotation=45)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```



```
[32]: user_question3 = "Given a component part, list all parents/subassemblies it is_
      ↪used in (multi-level)."
```

```
[33]: user_content5 = f"""Database: AdventureWorks
Task: Convert the following natural language request into SQL Server SQL.

Schema summary:
{database_schema}

Business context:
- This database represents a manufacturing company.

Important table hints:
- Work orders: Production.WorkOrder + Production.Product
- Inventory: Production.ProductInventory + Production.Location + Production.
  ↪Product
- Bill of Materials: Production.BillOfMaterials

Natural language request:
{user_question3}

Output requirements:
- Return valid JSON only.
"""
```

```
[34]: message3 = [
      {
        "role": "system",
```

```

        "content": system
    },
    {
        "role": "user",
        "content": user_content5    # MUST be a string
    }
]
message3

```

```

[34]: [{'role': 'system',
        'content': 'You are an expert data analyst and SQL generator for the
AdventureWorks database.\n\nYour task is to translate a user’s natural language
manufacturing or business question into a safe, correct SQL Server query and a
brief explanation.\n\nRules:\n1. Only generate read-only SQL.\n2. Allowed query
types are:\n    - SELECT\n    - Common Table Expressions (CTEs)\n    - ORDER BY\n    - GROUP BY\n    - HAVING\n    - JOINS\n    - window functions\n    - recursive CTEs
if needed\n3. Do not generate:\n    - INSERT\n    - UPDATE\n    - DELETE\n    -
DROP\n    - ALTER\n    - TRUNCATE\n    - EXEC\n    - MERGE\n    - stored
procedures\n4. The SQL dialect is Microsoft SQL Server (AdventureWorks).\n5.
Prefer fully qualified table names, for example:\n    Production.WorkOrder\n
Production.Product\n    Production.ProductInventory\n6. Use TOP for row limits,
not LIMIT.\n7. If the question is ambiguous, make the smallest reasonable
assumption and state it briefly.\n8. If the question cannot be answered from the
provided schema, say so clearly.\n9. Use joins only where supported by the
schema provided.\n10. Return output in valid JSON only.\n\nYou must return
exactly this JSON structure:\n\n{\n  "question_understanding": "short
restatement of the task",\n  "assumptions": ["assumption 1", "assumption 2"],\n
"sql": "single SQL Server query as a string",\n  "explanation": "brief
explanation of how the query works",\n  "result_shape": "what the output table
should contain",\n  "validation_checks": ["check 1", "check 2"],\n
"plot_suggestion": {\n    "recommended": true,\n    "plot_type":
"bar|line|pie|none",\n    "x": "column name or null",\n    "y": "column name or
null",\n    "title": "plot title or null"\n  }\n}\n\nAdditional requirements:\n-
The SQL must be executable as written.\n- Do not wrap SQL in markdown fences.\n-
If no chart is appropriate, set plot_suggestion.plot_type to "none" and other
plot fields to null.\n- If aggregation is used, ensure all non-aggregated
selected columns are included in GROUP BY.\n- If ranking is requested, prefer
ORDER BY and TOP, or window functions where appropriate.\n- If multi-level bill-
of-materials logic is requested, use a recursive CTE.'},
        {'role': 'user',
        'content': 'Database: AdventureWorks\nTask: Convert the following natural
language request into SQL Server SQL.\n\nSchema
summary:\nProduction.WorkOrder\n- WorkOrderID (PK)\n- ProductID (FK ->
Production.Product.ProductID)\n- OrderQty\n- StockedQty\n- ScrappedQty\n-
StartDate\n- EndDate\n- DueDate\n- ScrapReasonID\n\nProduction.Product\n-
ProductID (PK)\n- Name\n- ProductNumber\n- MakeFlag\n- FinishedGoodsFlag\n-
Color\n- SafetyStockLevel\n- ReorderPoint\n- StandardCost\n- ListPrice\n-

```

ProductSubcategoryID\n\nProduction.ProductInventory\n- ProductID (FK -> Production.Product.ProductID)\n- LocationID (FK -> Production.Location.LocationID)\n- Shelf\n- Bin\n- Quantity\n\nProduction.Location\n- LocationID (PK)\n- Name\n- CostRate\n- Availability\n\nProduction.WorkOrderRouting\n- WorkOrderID (FK -> Production.WorkOrder.WorkOrderID)\n- ProductID\n- OperationSequence\n- LocationID (FK -> Production.Location.LocationID)\n- ScheduledStartDate\n- ScheduledEndDate\n- ActualStartDate\n- ActualEndDate\n- PlannedCost\n- ActualCost\n\nProduction.BillOfMaterials\n- BillOfMaterialsID (PK)\n- ProductAssemblyID\n- ComponentID\n- StartDate\n- EndDate\n- UnitMeasureCode\n- BOMLevel\n- PerAssemblyQty\n\nProduction.TransactionHistory\n- TransactionID\n- ProductID\n- ReferenceOrderID\n- ReferenceOrderLineID\n- TransactionDate\n- TransactionType\n- Quantity\n- ActualCost\n\nBusiness context:\n- This database represents a manufacturing company.\n\nImportant table hints:\n- Work orders: Production.WorkOrder + Production.Product\n- Inventory: Production.ProductInventory + Production.Location + Production.Product\n- Bill of Materials: Production.BillOfMaterials\n\nNatural language request:\nGiven a component part, list all parents/subassemblies it is used in (multi-level).\n\nOutput requirements:\n- Return valid JSON only.\n'}}

```
[35]: response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=message3 # correct
)
print(response.choices[0].message.content)
```

```
{
  "question_understanding": "List all parent assemblies for a given component part using multi-level bill-of-materials logic.",
  "assumptions": ["The component part is identified by its ProductID, which needs to be specified.", "We will provide a generic ProductID in the query."],
  "sql": "WITH RecursiveBOM AS (\n    SELECT ProductAssemblyID, ComponentID, BOMLevel\n    FROM Production.BillOfMaterials\n    WHERE ComponentID = @ComponentID\n    UNION ALL\n    SELECT b.ProductAssemblyID, b.ComponentID, b.BOMLevel\n    FROM Production.BillOfMaterials b\n    INNER JOIN RecursiveBOM r\n    ON b.ComponentID = r.ProductAssemblyID\n)\nSELECT DISTINCT p.Name, p.ProductNumber\nFROM RecursiveBOM r\nJOIN Production.Product p ON r.ProductAssemblyID = p.ProductID;",
  "explanation": "The query uses a recursive Common Table Expression (CTE) to navigate the Bill of Materials table, starting with the specified component ID. It finds all parent assemblies that use this component directly. Then, it selects distinct names and product numbers of those assemblies from the Product table.",
  "result_shape": "The output table will contain two columns: Name (of the parent assembly) and ProductNumber (of the parent assembly).",
  "validation_checks": ["Ensure that the specified ProductID exists in the BillOfMaterials table.", "Check that the result set does not contain duplicates."],
}
```

```

    "plot_suggestion": {
        "recommended": false,
        "plot_type": "none",
        "x": null,
        "y": null,
        "title": null
    }
}

```

```

[36]: # Step 1: get a valid component id
component_id_query = """
SELECT TOP 1 ComponentID
FROM Production.BillOfMaterials
WHERE ComponentID IS NOT NULL
ORDER BY ComponentID;
"""

#component_id = pd.read_sql(component_id_query, conn).iloc[0, 0]
component_id = int(pd.read_sql(component_id_query, conn).iloc[0, 0])

# Step 2: recursive where-used query
query5 = """
WITH RecursiveBOM AS (
    SELECT
        b.ProductAssemblyID,
        b.ComponentID,
        1 AS HierarchyLevel
    FROM Production.BillOfMaterials AS b
    WHERE b.ComponentID = %s
        AND b.ProductAssemblyID IS NOT NULL

    UNION ALL

    SELECT
        b.ProductAssemblyID,
        b.ComponentID,
        r.HierarchyLevel + 1
    FROM Production.BillOfMaterials AS b
    INNER JOIN RecursiveBOM AS r
        ON b.ComponentID = r.ProductAssemblyID
    WHERE b.ProductAssemblyID IS NOT NULL
)
SELECT DISTINCT
    p.Name AS ParentAssemblyName,
    p.ProductID AS ParentAssemblyID,
    r.HierarchyLevel

```



```

FROM RecursiveBOM AS r
JOIN Production.Product AS p
    ON r.ProductAssemblyID = p.ProductID
ORDER BY
    r.HierarchyLevel,
    ParentAssemblyName;
"""

table5 = pd.read_sql(query5, conn, params=[component_id])
table5

```

C:\Users\emper\AppData\Local\Temp\ipykernel_55500\3335849953.py:10: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
component_id = int(pd.read_sql(component_id_query, conn).iloc[0, 0])
```

C:\Users\emper\AppData\Local\Temp\ipykernel_55500\3335849953.py:47: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
table5 = pd.read_sql(query5, conn, params=[component_id])
```

```
[36]:
```

	ParentAssemblyName	ParentAssemblyID	HierarchyLevel
0	HL Headset	807	1
1	LL Headset	805	1
2	ML Headset	806	1
3	Mountain-100 Black, 38	775	2
4	Mountain-100 Black, 42	776	2
..	...	...	...
95	Touring-3000 Yellow, 44	961	2
96	Touring-3000 Yellow, 50	962	2
97	Touring-3000 Yellow, 54	963	2
98	Touring-3000 Yellow, 58	964	2
99	Touring-3000 Yellow, 62	965	2

[100 rows x 3 columns]

Please note that the first output dis not have the **ParentAssemblyName** column. It was latter included while fixing the code

3 SQL Output Evaluation

3.1 Strenght

- It used the right operations logic, starting from the components and then scale up to the parent assemblies
- It used the most appropriate technique for a multi-level wher-used analysis which is the recursive CTE

- Correct table selection
- Consistent JSON output ## Weakness
- The query returns on the assembly ID **ProductAssemblyID**. It would be a better idea to return the assembly names in addition to the assembly ID

```
[39]: user_content6 = f"""
Database: AdventureWorks

Schema summary:
{database_schema}

Example:

Input:
Given a component part, list all parents/subassemblies it is used in
↳(multi-level).

Output:
{{
  "question_understanding": "List all parent assemblies and subassemblies in
↳which a given component part is used across multiple BOM levels.",
  "assumptions": [
    "The component part is identified by a SQL Server parameter named
↳@ComponentID.",
    "Parent assembly names should be returned in addition to assembly IDs.",
    "A recursive CTE is required to trace usage across multiple BOM levels."
  ],
  "sql": "WITH RecursiveBOM AS ( SELECT BOM.ProductAssemblyID, BOM.ComponentID,
↳1 AS BOMLevel FROM Production.BillOfMaterials AS BOM WHERE BOM.ComponentID =
↳@ComponentID UNION ALL SELECT BOM.ProductAssemblyID, BOM.ComponentID, RB.
↳BOMLevel + 1 FROM Production.BillOfMaterials AS BOM INNER JOIN RecursiveBOM
↳AS RB ON RB.ProductAssemblyID = BOM.ComponentID ) SELECT DISTINCT RB.
↳ProductAssemblyID, P.Name AS ParentAssemblyName, RB.BOMLevel FROM
↳RecursiveBOM AS RB JOIN Production.Product AS P ON RB.ProductAssemblyID = P.
↳ProductID ORDER BY RB.BOMLevel, ParentAssemblyName;",
  "explanation": "The query starts with the specified component part and finds
↳the assemblies that directly use it. It then recursively moves upward
↳through the Bill of Materials structure to identify higher-level parent
↳assemblies and subassemblies. Finally, it joins to the Product table to
↳return readable assembly names.",
  "result_shape": "A table containing ProductAssemblyID, ParentAssemblyName,
↳and BOMLevel for all parent assemblies and subassemblies that use the given
↳component.",
  "validation_checks": [
    "Verify that @ComponentID is provided before execution.",
    "Check that each returned ProductAssemblyID is a valid parent or ancestor
↳of the component.",
    "Confirm that duplicate parent assemblies are removed.",
  ]
}}
```

```

    "Verify that BOMLevel increases correctly as the recursion moves upward."
],
"plot_suggestion": {
  "recommended": false,
  "plot_type": "none",
  "x": null,
  "y": null,
  "title": null
}}
}}

---

Now answer:

{user_question3}

Return valid JSON only.
"""

```

```

[40]: message3 = [
    {
        "role": "system",
        "content": system
    },
    {
        "role": "user",
        "content": user_content6    # MUST be a string
    }
]
message3

```

```

[40]: [{ 'role': 'system',
        'content': 'You are an expert data analyst and SQL generator for the
AdventureWorks database.\n\nYour task is to translate a user's natural language
manufacturing or business question into a safe, correct SQL Server query and a
brief explanation.\n\nRules:\n1. Only generate read-only SQL.\n2. Allowed query
types are:\n    - SELECT\n    - Common Table Expressions (CTEs)\n    - ORDER BY\n    - GROUP BY\n    - HAVING\n    - JOINS\n    - window functions\n    - recursive CTEs
if needed\n3. Do not generate:\n    - INSERT\n    - UPDATE\n    - DELETE\n    -
DROP\n    - ALTER\n    - TRUNCATE\n    - EXEC\n    - MERGE\n    - stored
procedures\n4. The SQL dialect is Microsoft SQL Server (AdventureWorks).\n5.
Prefer fully qualified table names, for example:\n    Production.WorkOrder\n
Production.Product\n    Production.ProductInventory\n6. Use TOP for row limits,
not LIMIT.\n7. If the question is ambiguous, make the smallest reasonable
assumption and state it briefly.\n8. If the question cannot be answered from the
provided schema, say so clearly.\n9. Use joins only where supported by the

```

schema provided.\n10. Return output in valid JSON only.\n\nYou must return exactly this JSON structure:\n\n{\n "question_understanding": "short restatement of the task",\n "assumptions": ["assumption 1", "assumption 2"],\n "sql": "single SQL Server query as a string",\n "explanation": "brief explanation of how the query works",\n "result_shape": "what the output table should contain",\n "validation_checks": ["check 1", "check 2"],\n "plot_suggestion": {\n "recommended": true,\n "plot_type": "bar|line|pie|none",\n "x": "column name or null",\n "y": "column name or null",\n "title": "plot title or null"\n }\n}\n\nAdditional requirements:\n- The SQL must be executable as written.\n- Do not wrap SQL in markdown fences.\n- If no chart is appropriate, set plot_suggestion.plot_type to "none" and other plot fields to null.\n- If aggregation is used, ensure all non-aggregated selected columns are included in GROUP BY.\n- If ranking is requested, prefer ORDER BY and TOP, or window functions where appropriate.\n- If multi-level bill-of-materials logic is requested, use a recursive CTE.'},\n {'role': 'user',\n 'content': '\nDatabase: AdventureWorks\n\nSchema summary:\nProduction.WorkOrder\n- WorkOrderID (PK)\n- ProductID (FK -> Production.Product.ProductID)\n- OrderQty\n- StockedQty\n- ScrappedQty\n- StartDate\n- EndDate\n- DueDate\n- ScrapReasonID\n\nProduction.Product\n- ProductID (PK)\n- Name\n- ProductNumber\n- MakeFlag\n- FinishedGoodsFlag\n- Color\n- SafetyStockLevel\n- ReorderPoint\n- StandardCost\n- ListPrice\n- ProductSubcategoryID\n\nProduction.ProductInventory\n- ProductID (FK -> Production.Product.ProductID)\n- LocationID (FK -> Production.Location.LocationID)\n- Shelf\n- Bin\n- Quantity\n\nProduction.Location\n- LocationID (PK)\n- Name\n- CostRate\n- Availability\n\nProduction.WorkOrderRouting\n- WorkOrderID (FK -> Production.WorkOrder.WorkOrderID)\n- ProductID\n- OperationSequence\n- LocationID (FK -> Production.Location.LocationID)\n- ScheduledStartDate\n- ScheduledEndDate\n- ActualStartDate\n- ActualEndDate\n- PlannedCost\n- ActualCost\n\nProduction.BillOfMaterials\n- BillOfMaterialsID (PK)\n- ProductAssemblyID\n- ComponentID\n- StartDate\n- EndDate\n- UnitMeasureCode\n- BOMLevel\n- PerAssemblyQty\n\nProduction.TransactionHistory\n- TransactionID\n- ProductID\n- ReferenceOrderID\n- ReferenceOrderLineID\n- TransactionDate\n- TransactionType\n- Quantity\n- ActualCost\n\nExample:\n\nInput:\nGiven a component part, list all parents/subassemblies it is used in (multi-level).\n\nOutput:\n{\n "question_understanding": "List all parent assemblies and subassemblies in which a given component part is used across multiple BOM levels.",\n "assumptions": [\n "The component part is identified by a SQL Server parameter named @ComponentID.",\n "Parent assembly names should be returned in addition to assembly IDs.",\n "A recursive CTE is required to trace usage across multiple BOM levels."],\n "sql": "WITH RecursiveBOM AS (\n SELECT BOM.ProductAssemblyID, BOM.ComponentID, 1 AS BOMLevel FROM\n Production.BillOfMaterials AS BOM WHERE BOM.ComponentID = @ComponentID UNION ALL\n SELECT BOM.ProductAssemblyID, BOM.ComponentID, RB.BOMLevel + 1 FROM\n Production.BillOfMaterials AS BOM INNER JOIN RecursiveBOM AS RB ON\n RB.ProductAssemblyID = BOM.ComponentID) SELECT DISTINCT RB.ProductAssemblyID,

```
P.Name AS ParentAssemblyName, RB.BOMLevel FROM RecursiveBOM AS RB JOIN
Production.Product AS P ON RB.ProductAssemblyID = P.ProductID ORDER BY
RB.BOMLevel, ParentAssemblyName;\",\n  "explanation": "The query starts with the
specified component part and finds the assemblies that directly use it. It then
recursively moves upward through the Bill of Materials structure to identify
higher-level parent assemblies and subassemblies. Finally, it joins to the
Product table to return readable assembly names.",\n  "result_shape": "A table
containing ProductAssemblyID, ParentAssemblyName, and BOMLevel for all parent
assemblies and subassemblies that use the given component.",\n
"validation_checks": [\n    "Verify that @ComponentID is provided before
execution.",\n    "Check that each returned ProductAssemblyID is a valid parent
or ancestor of the component.",\n    "Confirm that duplicate parent assemblies
are removed.",\n    "Verify that BOMLevel increases correctly as the recursion
moves upward.",\n  ],\n  "plot_suggestion": {\n    "recommended": false,\n    "plot_type": "none",\n    "x": null,\n    "y": null,\n    "title": null\n  }\n}\n\n---\n\nNow answer:\n\nGiven a component part, list all
parents/subassemblies it is used in (multi-level).\n\nReturn valid JSON
only.\n'}]
```

```
[41]: response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=message3 # correct
)
print(response.choices[0].message.content)
```

```
{
  "question_understanding": "List all parent assemblies and subassemblies in
which a given component part is used across multiple BOM levels.",
  "assumptions": [
    "The component part is identified by a SQL Server parameter named
@ComponentID.",
    "Parent assembly names should be returned in addition to assembly IDs.",
    "A recursive CTE is required to trace usage across multiple BOM levels."
  ],
  "sql": "WITH RecursiveBOM AS ( SELECT BOM.ProductAssemblyID, BOM.ComponentID,
1 AS BOMLevel FROM Production.BillOfMaterials AS BOM WHERE BOM.ComponentID =
@ComponentID UNION ALL SELECT BOM.ProductAssemblyID, BOM.ComponentID,
RB.BOMLevel + 1 FROM Production.BillOfMaterials AS BOM INNER JOIN RecursiveBOM
AS RB ON RB.ProductAssemblyID = BOM.ComponentID ) SELECT DISTINCT
RB.ProductAssemblyID, P.Name AS ParentAssemblyName, RB.BOMLevel FROM
RecursiveBOM AS RB JOIN Production.Product AS P ON RB.ProductAssemblyID =
P.ProductID ORDER BY RB.BOMLevel, ParentAssemblyName;",
  "explanation": "The query starts with the specified component part and finds
the assemblies that directly use it. It then recursively moves upward through
the Bill of Materials structure to identify higher-level parent assemblies and
subassemblies. Finally, it joins to the Product table to return readable
assembly names.",
  "result_shape": "A table containing ProductAssemblyID, ParentAssemblyName, and
```

```

BOMLevel for all parent assemblies and subassemblies that use the given
component.",
  "validation_checks": [
    "Verify that @ComponentID is provided before execution.",
    "Check that each returned ProductAssemblyID is a valid parent or ancestor of
the component.",
    "Confirm that duplicate parent assemblies are removed.",
    "Verify that BOMLevel increases correctly as the recursion moves upward."
  ],
  "plot_suggestion": {
    "recommended": false,
    "plot_type": "none",
    "x": null,
    "y": null,
    "title": null
  }
}

```

4 Additional Improvement

- Add filter to the BOM row to avoid obsolete or inactive relationship

```

[43]: user_content6 = f"""
Database: AdventureWorks

Schema summary:
{database_schema}

Example:

Input:
Given a component part, list all parents/subassemblies it is used in_
↳(multi-level).

Output:
{{
  "question_understanding": "List all parent assemblies and subassemblies in_
↳which a given component part is used across multiple BOM levels, considering_
↳only current BOM relationships.",
  "assumptions": [
    "The component part is identified by a SQL Server parameter named_
↳@ComponentID.",
    "Only active Bill of Materials relationships should be included, so rows_
↳with EndDate not null are excluded.",
    "Parent assembly names should be returned in addition to assembly IDs.",
    "A recursive CTE is required to trace usage across multiple BOM levels."
  ],

```

```

    "sql": "WITH RecursiveBOM AS ( SELECT BOM.ProductAssemblyID, BOM.ComponentID,
    ↳1 AS BOMLevel FROM Production.BillofMaterials AS BOM WHERE BOM.ComponentID =
    ↳@ComponentID AND BOM.EndDate IS NULL UNION ALL SELECT BOM.ProductAssemblyID,
    ↳BOM.ComponentID, RB.BOMLevel + 1 FROM Production.BillofMaterials AS BOM
    ↳INNER JOIN RecursiveBOM AS RB ON RB.ProductAssemblyID = BOM.ComponentID
    ↳WHERE BOM.EndDate IS NULL ) SELECT DISTINCT RB.ProductAssemblyID, P.Name AS
    ↳ParentAssemblyName, RB.BOMLevel FROM RecursiveBOM AS RB JOIN Production.
    ↳Product AS P ON RB.ProductAssemblyID = P.ProductID ORDER BY RB.BOMLevel,
    ↳ParentAssemblyName;",
    "explanation": "The query starts from the specified component and finds
    ↳assemblies that currently use it by filtering the Bill of Materials to
    ↳active rows where EndDate is null. It then recursively moves upward through
    ↳higher BOM levels to identify parent assemblies and subassemblies, and joins
    ↳to the Product table to return readable parent assembly names.",
    "result_shape": "A table containing ProductAssemblyID, ParentAssemblyName,
    ↳and BOMLevel for all active parent assemblies and subassemblies that use the
    ↳given component.",
    "validation_checks": [
        "Verify that @ComponentID is provided before execution.",
        "Check that only BOM rows with EndDate IS NULL are included.",
        "Check that each returned ProductAssemblyID is a valid current parent or
    ↳ancestor of the component.",
        "Confirm that duplicate parent assemblies are removed.",
        "Verify that BOMLevel increases correctly as the recursion moves upward."
    ],
    "plot_suggestion": {
        "recommended": false,
        "plot_type": "none",
        "x": null,
        "y": null,
        "title": null
    }
}
}

---

Now answer:

{user_question3}

Return valid JSON only.
"""

```

```

[44]: message3 = [
    {
        "role": "system",
        "content": system
    }
]

```

```

    },
    {
        "role": "user",
        "content": user_content6    # MUST be a string
    }
]
message3

```

```

[44]: [{ 'role': 'system',
        'content': 'You are an expert data analyst and SQL generator for the
AdventureWorks database.\n\nYour task is to translate a user's natural language
manufacturing or business question into a safe, correct SQL Server query and a
brief explanation.\n\nRules:\n1. Only generate read-only SQL.\n2. Allowed query
types are:\n    - SELECT\n    - Common Table Expressions (CTEs)\n    - ORDER BY\n    - GROUP BY\n    - HAVING\n    - JOINS\n    - window functions\n    - recursive CTEs
if needed\n3. Do not generate:\n    - INSERT\n    - UPDATE\n    - DELETE\n    -
DROP\n    - ALTER\n    - TRUNCATE\n    - EXEC\n    - MERGE\n    - stored
procedures\n4. The SQL dialect is Microsoft SQL Server (AdventureWorks).\n5.
Prefer fully qualified table names, for example:\n    Production.WorkOrder\n
Production.Product\n    Production.ProductInventory\n6. Use TOP for row limits,
not LIMIT.\n7. If the question is ambiguous, make the smallest reasonable
assumption and state it briefly.\n8. If the question cannot be answered from the
provided schema, say so clearly.\n9. Use joins only where supported by the
schema provided.\n10. Return output in valid JSON only.\n\nYou must return
exactly this JSON structure:\n\n{\n  "question_understanding": "short
restatement of the task",\n  "assumptions": ["assumption 1", "assumption 2"],\n
"sql": "single SQL Server query as a string",\n  "explanation": "brief
explanation of how the query works",\n  "result_shape": "what the output table
should contain",\n  "validation_checks": ["check 1", "check 2"],\n
"plot_suggestion": {\n    "recommended": true,\n    "plot_type":
"bar|line|pie|none",\n    "x": "column name or null",\n    "y": "column name or
null",\n    "title": "plot title or null"\n  }\n}\n\nAdditional requirements:\n-
The SQL must be executable as written.\n- Do not wrap SQL in markdown fences.\n-
If no chart is appropriate, set plot_suggestion.plot_type to "none" and other
plot fields to null.\n- If aggregation is used, ensure all non-aggregated
selected columns are included in GROUP BY.\n- If ranking is requested, prefer
ORDER BY and TOP, or window functions where appropriate.\n- If multi-level bill-
of-materials logic is requested, use a recursive CTE.'},
    { 'role': 'user',
      'content': '\nDatabase: AdventureWorks\n\nSchema
summary:\nProduction.WorkOrder\n- WorkOrderID (PK)\n- ProductID (FK ->
Production.Product.ProductID)\n- OrderQty\n- StockedQty\n- ScrappedQty\n-
StartDate\n- EndDate\n- DueDate\n- ScrapReasonID\n\nProduction.Product\n-
ProductID (PK)\n- Name\n- ProductNumber\n- MakeFlag\n- FinishedGoodsFlag\n-
Color\n- SafetyStockLevel\n- ReorderPoint\n- StandardCost\n- ListPrice\n-
ProductSubcategoryID\n\nProduction.ProductInventory\n- ProductID (FK ->
Production.Product.ProductID)\n- LocationID (FK ->

```



```

Production.Location.LocationID)\n- Shelf\n- Bin\n-
Quantity\n\nProduction.Location\n- LocationID (PK)\n- Name\n- CostRate\n-
Availability\n\nProduction.WorkOrderRouting\n- WorkOrderID (FK ->
Production.WorkOrder.WorkOrderID)\n- ProductID\n- OperationSequence\n-
LocationID (FK -> Production.Location.LocationID)\n- ScheduledStartDate\n-
ScheduledEndDate\n- ActualStartDate\n- ActualEndDate\n- PlannedCost\n-
ActualCost\n\nProduction.BillOfMaterials\n- BillOfMaterialsID (PK)\n-
ProductAssemblyID\n- ComponentID\n- StartDate\n- EndDate\n- UnitMeasureCode\n-
BOMLevel\n- PerAssemblyQty\n\nProduction.TransactionHistory\n- TransactionID\n-
ProductID\n- ReferenceOrderID\n- ReferenceOrderLineID\n- TransactionDate\n-
TransactionType\n- Quantity\n- ActualCost\n\nExample:\n\nInput:\nGiven a
component part, list all parents/subassemblies it is used in (multi-
level).\nOutput:\n{\n  "question_understanding": "List all parent assemblies and
subassemblies in which a given component part is used across multiple BOM
levels, considering only current BOM relationships.",\n  "assumptions": [\n
"The component part is identified by a SQL Server parameter named
@ComponentID.",\n    "Only active Bill of Materials relationships should be
included, so rows with EndDate not null are excluded.",\n    "Parent assembly
names should be returned in addition to assembly IDs.",\n    "A recursive CTE is
required to trace usage across multiple BOM levels." ] ,\n  "sql": "WITH
RecursiveBOM AS ( SELECT BOM.ProductAssemblyID, BOM.ComponentID, 1 AS BOMLevel
FROM Production.BillOfMaterials AS BOM WHERE BOM.ComponentID = @ComponentID AND
BOM.EndDate IS NULL UNION ALL SELECT BOM.ProductAssemblyID, BOM.ComponentID,
RB.BOMLevel + 1 FROM Production.BillOfMaterials AS BOM INNER JOIN RecursiveBOM
AS RB ON RB.ProductAssemblyID = BOM.ComponentID WHERE BOM.EndDate IS NULL )
SELECT DISTINCT RB.ProductAssemblyID, P.Name AS ParentAssemblyName, RB.BOMLevel
FROM RecursiveBOM AS RB JOIN Production.Product AS P ON RB.ProductAssemblyID =
P.ProductID ORDER BY RB.BOMLevel, ParentAssemblyName;",\n  "explanation": "The
query starts from the specified component and finds assemblies that currently
use it by filtering the Bill of Materials to active rows where EndDate is null.
It then recursively moves upward through higher BOM levels to identify parent
assemblies and subassemblies, and joins to the Product table to return readable
parent assembly names.",\n  "result_shape": "A table containing
ProductAssemblyID, ParentAssemblyName, and BOMLevel for all active parent
assemblies and subassemblies that use the given component.",\n  "validation_checks": [\n    "Verify that @ComponentID is provided before
execution.",\n    "Check that only BOM rows with EndDate IS NULL are
included.",\n    "Check that each returned ProductAssemblyID is a valid current
parent or ancestor of the component.",\n    "Confirm that duplicate parent
assemblies are removed.",\n    "Verify that BOMLevel increases correctly as the
recursion moves upward." ] ,\n  "plot_suggestion": {\n    "recommended":
false,\n    "plot_type": "none",\n    "x": null,\n    "y": null,\n    "title":
null\n  }\n}\n\n---\n\nNow answer:\n\nGiven a component part, list all
parents/subassemblies it is used in (multi-level).\n\nReturn valid JSON
only.\n'}]

```

```
[45]: response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=message3 # correct
)
print(response.choices[0].message.content)

{
  "question_understanding": "List all parent assemblies and subassemblies in which a given component part is used across multiple BOM levels, considering only current BOM relationships.",
  "assumptions": [
    "The component part is identified by a SQL Server parameter named @ComponentID.",
    "Only active Bill of Materials relationships should be included, so rows with EndDate not null are excluded.",
    "Parent assembly names should be returned in addition to assembly IDs.",
    "A recursive CTE is required to trace usage across multiple BOM levels."
  ],
  "sql": "WITH RecursiveBOM AS ( SELECT BOM.ProductAssemblyID, BOM.ComponentID, 1 AS BOMLevel FROM Production.BillofMaterials AS BOM WHERE BOM.ComponentID = @ComponentID AND BOM.EndDate IS NULL UNION ALL SELECT BOM.ProductAssemblyID, BOM.ComponentID, RB.BOMLevel + 1 FROM Production.BillofMaterials AS BOM INNER JOIN RecursiveBOM AS RB ON RB.ProductAssemblyID = BOM.ComponentID WHERE BOM.EndDate IS NULL ) SELECT DISTINCT RB.ProductAssemblyID, P.Name AS ParentAssemblyName, RB.BOMLevel FROM RecursiveBOM AS RB JOIN Production.Product AS P ON RB.ProductAssemblyID = P.ProductID ORDER BY RB.BOMLevel, ParentAssemblyName;",
  "explanation": "The query starts from the specified component and finds assemblies that currently use it by filtering the Bill of Materials to active rows where EndDate is null. It then recursively moves upward through higher BOM levels to identify parent assemblies and subassemblies, and joins to the Product table to return readable parent assembly names.",
  "result_shape": "A table containing ProductAssemblyID, ParentAssemblyName, and BOMLevel for all active parent assemblies and subassemblies that use the given component.",
  "validation_checks": [
    "Verify that @ComponentID is provided before execution.",
    "Check that only BOM rows with EndDate IS NULL are included.",
    "Check that each returned ProductAssemblyID is a valid current parent or ancestor of the component.",
    "Confirm that duplicate parent assemblies are removed.",
    "Verify that BOMLevel increases correctly as the recursion moves upward."
  ],
  "plot_suggestion": {
    "recommended": false,
    "plot_type": "none",
    "x": null,
    "y": null,
  }
}
```

```

        "title": null
    }
}

```

```

[46]: query6 = """
WITH RecursiveBOM AS (
    SELECT
        BOM.ProductAssemblyID,
        BOM.ComponentID,
        1 AS BOMLevel
    FROM Production.BillofMaterials AS BOM
    WHERE BOM.ComponentID = %d
        AND BOM.EndDate IS NULL
        AND BOM.ProductAssemblyID IS NOT NULL

    UNION ALL

    SELECT
        BOM.ProductAssemblyID,
        BOM.ComponentID,
        RB.BOMLevel + 1
    FROM Production.BillofMaterials AS BOM
    INNER JOIN RecursiveBOM AS RB
        ON RB.ProductAssemblyID = BOM.ComponentID
    WHERE BOM.EndDate IS NULL
        AND BOM.ProductAssemblyID IS NOT NULL
)
SELECT DISTINCT
    RB.ProductAssemblyID,
    P.Name AS ParentAssemblyName,
    RB.BOMLevel
FROM RecursiveBOM AS RB
JOIN Production.Product AS P
    ON RB.ProductAssemblyID = P.ProductID
ORDER BY
    RB.BOMLevel,
    ParentAssemblyName;
"""

component_id_query = """
SELECT TOP 1 ComponentID
FROM Production.BillofMaterials
WHERE ComponentID IS NOT NULL
    AND EndDate IS NULL
ORDER BY ComponentID;
"""

```

```

component_id = int(pd.read_sql(component_id_query, conn).iloc[0, 0])

final_query6 = query5 % component_id
table6 = pd.read_sql(final_query6, conn)
table6
#table6 = pd.read_sql(query6, conn)
#table6

```

C:\Users\emper\AppData\Local\Temp\ipykernel_55500\1136698807.py:44: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
component_id = int(pd.read_sql(component_id_query, conn).iloc[0, 0])
```

C:\Users\emper\AppData\Local\Temp\ipykernel_55500\1136698807.py:47: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
table6 = pd.read_sql(final_query6, conn)
```

```
[46]:
```

	ParentAssemblyName	ParentAssemblyID	HierarchyLevel
0	HL Headset	807	1
1	LL Headset	805	1
2	ML Headset	806	1
3	Mountain-100 Black, 38	775	2
4	Mountain-100 Black, 42	776	2
..	...	...	...
95	Touring-3000 Yellow, 44	961	2
96	Touring-3000 Yellow, 50	962	2
97	Touring-3000 Yellow, 54	963	2
98	Touring-3000 Yellow, 58	964	2
99	Touring-3000 Yellow, 62	965	2

[100 rows x 3 columns]

```
[47]: #table6.to_csv("parents_subassembly_df.csv", index=False)
```

```
[243]: parent_sub_assemb = pd.read_csv("parents_subassembly_df.csv")
parent_sub_assemb.head()
```

```
[243]:
```

	ParentAssemblyName	ParentAssemblyID	HierarchyLevel
0	HL Headset	807	1
1	LL Headset	805	1
2	ML Headset	806	1
3	Mountain-100 Black, 38	775	2
4	Mountain-100 Black, 42	776	2

```
[ ]:
```

5 Question 4

```
[49]: user_question4 = """Scrap risk classifier Question:
"Can we predict whether a work order will incur any scrap?"
Target:
• Binary: scrap occurred (yes/no) on any routing row for that work order_
  ↳Features:
• Product, category
• Locations visited
• Operation count
• Quantity
• Vendor mix of component parts (optional) Model:
• Logistic regression baseline + tree-based model Metrics:
• ROC-AUC + precision/recall (scrap is often imbalanced)
"""

[50]: user_content7 = f"""Database: AdventureWorks
Task: Convert the following natural language request into SQL Server SQL.

Schema summary:
{database_schema}

Business context:
- This database represents a manufacturing company.

Important table hints:
- Work orders: Production.WorkOrder + Production.Product
- Inventory: Production.ProductInventory + Production.Location + Production.
  ↳Product
- Routing: Production.WorkOrderRouting + Production.Location
- Scrap modelling table: Production.WorkOrder + Production.WorkOrderRouting +_
  ↳Production.Product

Natural language request:
{user_question4}

Output requirements:
- Return valid JSON only.
"""

[51]: message4 = [
    {
        "role": "system",
        "content": system
    },
    {
        "role": "user",
```

```

        "content": user_content7    # MUST be a string
    }
]
message4

```

```

[51]: [{'role': 'system',
        'content': 'You are an expert data analyst and SQL generator for the
AdventureWorks database.\n\nYour task is to translate a user's natural language
manufacturing or business question into a safe, correct SQL Server query and a
brief explanation.\n\nRules:\n1. Only generate read-only SQL.\n2. Allowed query
types are:\n    - SELECT\n    - Common Table Expressions (CTEs)\n    - ORDER BY\n    - GROUP BY\n    - HAVING\n    - JOINS\n    - window functions\n    - recursive CTEs
if needed\n3. Do not generate:\n    - INSERT\n    - UPDATE\n    - DELETE\n    -
DROP\n    - ALTER\n    - TRUNCATE\n    - EXEC\n    - MERGE\n    - stored
procedures\n4. The SQL dialect is Microsoft SQL Server (AdventureWorks).\n5.
Prefer fully qualified table names, for example:\n    Production.WorkOrder\n
Production.Product\n    Production.ProductInventory\n6. Use TOP for row limits,
not LIMIT.\n7. If the question is ambiguous, make the smallest reasonable
assumption and state it briefly.\n8. If the question cannot be answered from the
provided schema, say so clearly.\n9. Use joins only where supported by the
schema provided.\n10. Return output in valid JSON only.\n\nYou must return
exactly this JSON structure:\n\n{\n  "question_understanding": "short
restatement of the task",\n  "assumptions": ["assumption 1", "assumption 2"],\n
"sql": "single SQL Server query as a string",\n  "explanation": "brief
explanation of how the query works",\n  "result_shape": "what the output table
should contain",\n  "validation_checks": ["check 1", "check 2"],\n
"plot_suggestion": {\n    "recommended": true,\n    "plot_type":
"bar|line|pie|none",\n    "x": "column name or null",\n    "y": "column name or
null",\n    "title": "plot title or null"\n  }\n}\n\nAdditional requirements:\n-
The SQL must be executable as written.\n- Do not wrap SQL in markdown fences.\n-
If no chart is appropriate, set plot_suggestion.plot_type to "none" and other
plot fields to null.\n- If aggregation is used, ensure all non-aggregated
selected columns are included in GROUP BY.\n- If ranking is requested, prefer
ORDER BY and TOP, or window functions where appropriate.\n- If multi-level bill-
of-materials logic is requested, use a recursive CTE.'},
        {'role': 'user',
        'content': 'Database: AdventureWorks\nTask: Convert the following natural
language request into SQL Server SQL.\n\nSchema
summary:\nProduction.WorkOrder\n- WorkOrderID (PK)\n- ProductID (FK ->
Production.Product.ProductID)\n- OrderQty\n- StockedQty\n- ScrappedQty\n-
StartDate\n- EndDate\n- DueDate\n- ScrapReasonID\n\nProduction.Product\n-
ProductID (PK)\n- Name\n- ProductNumber\n- MakeFlag\n- FinishedGoodsFlag\n-
Color\n- SafetyStockLevel\n- ReorderPoint\n- StandardCost\n- ListPrice\n-
ProductSubcategoryID\n\nProduction.ProductInventory\n- ProductID (FK ->
Production.Product.ProductID)\n- LocationID (FK ->
Production.Location.LocationID)\n- Shelf\n- Bin\n-
Quantity\n\nProduction.Location\n- LocationID (PK)\n- Name\n- CostRate\n-

```

Availability\n\nProduction.WorkOrderRouting\n- WorkOrderID (FK -> Production.WorkOrder.WorkOrderID)\n- ProductID\n- OperationSequence\n- LocationID (FK -> Production.Location.LocationID)\n- ScheduledStartDate\n- ScheduledEndDate\n- ActualStartDate\n- ActualEndDate\n- PlannedCost\n- ActualCost\n\nProduction.BillOfMaterials\n- BillOfMaterialsID (PK)\n- ProductAssemblyID\n- ComponentID\n- StartDate\n- EndDate\n- UnitMeasureCode\n- BOMLevel\n- PerAssemblyQty\n\nProduction.TransactionHistory\n- TransactionID\n- ProductID\n- ReferenceOrderID\n- ReferenceOrderLineID\n- TransactionDate\n- TransactionType\n- Quantity\n- ActualCost\n\nBusiness context:\n- This database represents a manufacturing company.\n\nImportant table hints:\n- Work orders: Production.WorkOrder + Production.Product\n- Inventory: Production.ProductInventory + Production.Location + Production.Product\n- Routing: Production.WorkOrderRouting + Production.Location\n- Scrap modelling table: Production.WorkOrder + Production.WorkOrderRouting + Production.Product\n\nNatural language request:\nScrap risk classifier
Question:\n"Can we predict whether a work order will incur any scrap?"
\nTarget:\n• Binary: scrap occurred (yes/no) on any routing row for that work order
Features:\n• Product, category\n• Locations visited\n• Operation count\n• Quantity\n• Vendor mix of component parts (optional)
Model:\n• Logistic regression baseline + tree-based model
Metrics:\n• ROC-AUC + precision/recall (scrap is often imbalanced)\n\nOutput requirements:\n- Return valid JSON only.\n']}]

```
[52]: response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=message4 # correct
)
print(response.choices[0].message.content)
```

```
{
  "question_understanding": "Determine if a work order will incur any scrap based on routing information.",
  "assumptions": [
    "The work order is related to possible scrap occurrences across all its routing entries.",
    "Vendor mix information is not available in the schema."
  ],
  "sql": "WITH WorkOrderScrap AS (SELECT wo.WorkOrderID, MAX(CASE WHEN wo.ScrappedQty > 0 THEN 1 ELSE 0 END) AS ScrapOccurred, p.Name AS ProductName, p.ProductSubcategoryID, wr.LocationID, COUNT(wr.OperationSequence) AS OperationCount, SUM(wo.OrderQty) AS TotalQuantity FROM Production.WorkOrder AS wo JOIN Production.Product AS p ON wo.ProductID = p.ProductID JOIN Production.WorkOrderRouting AS wr ON wo.WorkOrderID = wr.WorkOrderID GROUP BY wo.WorkOrderID, p.Name, p.ProductSubcategoryID, wr.LocationID) SELECT WorkOrderID, ScrapOccurred, ProductName, ProductSubcategoryID, LocationID, OperationCount, TotalQuantity FROM WorkOrderScrap",
  "explanation": "The query uses a Common Table Expression (CTE) to calculate whether any scrap occurred for each work order by checking if the ScrappedQty is
```

greater than zero. It also includes features such as product name, product subcategory ID, location ID from the routing details, operation count, and total quantity of the work order. Finally, it selects the computed fields for output.",

```
"result_shape": "The output table will contain columns for WorkOrderID,
ScrapOccurred (binary), ProductName, ProductSubcategoryID, LocationID,
OperationCount, and TotalQuantity.",
"validation_checks": ["Check if WorkOrderID is unique in the output", "Verify
that ScrapOccurred correctly reflects the scrap quantity for each work order"],
"plot_suggestion": {
  "recommended": true,
  "plot_type": "bar",
  "x": "ProductName",
  "y": "TotalQuantity",
  "title": "Total Quantity of Scrap by Product"
}
}
```

6 SQL Output Evaluation

6.1 Strength

- Correct target grain, e.g. one row per **WorkOrderID**
- binary **scrap_flag**
- product and quantity feature,
- routing-derived location feature ## Weakness
- Wrong plot suggestion- the bar chart is usually not a good analytical plot
- Wrong operation count, i.e. ****COUNT(*) OVER (PARTITION BY WorkOrderID) AS operation_count****. The WorkOrderDetails already has one row per work order, it will always return 1 which may not be the exact number of routing operation
- There is not column for the location visited

```
[54]: query7 = """
WITH WorkOrderDetails AS (
  SELECT
    wo.WorkOrderID,
    CASE
      WHEN wo.ScrappedQty > 0 THEN 1
      ELSE 0
    END AS scrap_flag,
    wo.ProductID,
    CONCAT(p.Name, ' (' , p.ProductNumber, ')') AS product_descriptor,
    p.ProductSubcategoryID,
    wo.OrderQty,
    COUNT(DISTINCT wr.LocationID) AS distinct_locations
  FROM Production.WorkOrder AS wo
  JOIN Production.Product AS p
    ON wo.ProductID = p.ProductID
```



```

LEFT JOIN Production.WorkOrderRouting AS wr
    ON wo.WorkOrderID = wr.WorkOrderID
GROUP BY
    wo.WorkOrderID,
    wo.ScrapQty,
    wo.ProductID,
    p.Name,
    p.ProductNumber,
    p.ProductSubcategoryID,
    wo.OrderQty
)
SELECT
    WorkOrderID,
    scrap_flag,
    ProductID,
    product_descriptor,
    ProductSubcategoryID,
    OrderQty,
    distinct_locations,
    COUNT(*) OVER (PARTITION BY WorkOrderID) AS operation_count
FROM WorkOrderDetails;
"""
table7= pd.read_sql(query7, conn)
table7

```

C:\Users\emper\AppData\Local\Temp\ipykernel_55500\3759809544.py:39: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
table7= pd.read_sql(query7, conn)
```

```
[54]:
```

	WorkOrderID	scrap_flag	ProductID	\
0	1	0	722	
1	2	0	725	
2	3	0	726	
3	4	0	729	
4	5	0	730	
...	...	...	...	
72586	72587	0	804	
72587	72588	0	316	
72588	72589	0	331	
72589	72590	0	350	
72590	72591	0	531	

	product_descriptor	ProductSubcategoryID	OrderQty	\
0	LL Road Frame - Black, 58 (FR-R38B-58)	14.0	8	
1	LL Road Frame - Red, 44 (FR-R38R-44)	14.0	15	

2	LL Road Frame - Red, 48 (FR-R38R-48)	14.0	9
3	LL Road Frame - Red, 60 (FR-R38R-60)	14.0	16
4	LL Road Frame - Red, 62 (FR-R38R-62)	14.0	14
...	...	...	...
72586	HL Fork (FK-9939)	10.0	19
72587	Blade (BL-2036)	NaN	52
72588	Fork End (FE-3760)	NaN	52
72589	Fork Crown (FC-3654)	NaN	26
72590	Steerer (SR-2098)	NaN	26

	distinct_locations	operation_count
0	0	1
1	0	1
2	0	1
3	0	1
4	0	1
...	...	...
72586	2	1
72587	0	1
72588	0	1
72589	0	1
72590	0	1

[72591 rows x 8 columns]

```
[55]: #table7.to_csv("scrap_df.csv", index=False)
```

```
[56]: user_content7 = f"""
Database: AdventureWorks
Task: Convert the following natural language request into SQL Server SQL.
Schema summary:
{database_schema}

Example:

Input:
Create a work-order-level modelling table for a scrap risk classifier.
Return one row per WorkOrderID with:
- scrap_flag
- ProductID
- product descriptor
- category/subcategory if available
- OrderQty
- operation_count
- distinct_locations

Output:
```

```

{{
  "question_understanding": "Create a clean work-order-level modelling table,
  ↪for predicting whether a work order will incur any scrap.",
  "assumptions": [
    "The target is binary: scrap_flag = 1 if ScrappedQty > 0, otherwise 0.",
    "Each output row should represent one work order.",
    "The product descriptor is built from product name and product number.",
    "ProductSubcategoryID is used as the available category-related feature."
  ],
  "sql": "WITH WorkOrderBase AS ( SELECT wo.WorkOrderID, CASE WHEN wo.
  ↪ScrappedQty > 0 THEN 1 ELSE 0 END AS scrap_flag, wo.ProductID, wo.OrderQty,
  ↪FROM Production.WorkOrder AS wo ), RoutingFeatures AS ( SELECT wr.
  ↪WorkOrderID, COUNT(*) AS operation_count, COUNT(DISTINCT wr.LocationID) AS
  ↪distinct_locations FROM Production.WorkOrderRouting AS wr GROUP BY wr.
  ↪WorkOrderID ), ProductFeatures AS ( SELECT p.ProductID, CONCAT(p.Name, ' (',
  ↪p.ProductNumber, ')') AS product_descriptor, p.ProductSubcategoryID FROM
  ↪Production.Product AS p ) SELECT wb.WorkOrderID, wb.scrap_flag, wb.
  ↪ProductID, pf.product_descriptor, pf.ProductSubcategoryID, wb.OrderQty, rf.
  ↪operation_count, rf.distinct_locations FROM WorkOrderBase AS wb JOIN
  ↪ProductFeatures AS pf ON wb.ProductID = pf.ProductID LEFT JOIN
  ↪RoutingFeatures AS rf ON wb.WorkOrderID = rf.WorkOrderID;",
  "explanation": "The query builds a modelling table at work-order level. It
  ↪derives a binary scrap target from WorkOrder.ScrappedQty, aggregates routing
  ↪records to compute operation count and number of distinct locations, and
  ↪joins product details to provide candidate predictive features.",
  "result_shape": "One row per WorkOrderID containing scrap_flag, ProductID,
  ↪product_descriptor, ProductSubcategoryID, OrderQty, operation_count, and
  ↪distinct_locations.",
  "validation_checks": [
    "Verify that each WorkOrderID appears only once in the final output.",
    "Verify that scrap_flag is 1 when ScrappedQty > 0 and 0 otherwise.",
    "Verify that operation_count equals the number of WorkOrderRouting rows for
  ↪each work order.",
    "Verify that distinct_locations equals the number of unique LocationID
  ↪values per work order."
  ],
  "plot_suggestion": {
    "recommended": false,
    "plot_type": "none",
    "x": null,
    "y": null,
    "title": null
  }
}}

```

Business context:

- This database represents a manufacturing company.

Important table hints:

- Work orders: Production.WorkOrder + Production.Product
- Inventory: Production.ProductInventory + Production.Location + Production.
↳Product
- Routing: Production.WorkOrderRouting + Production.Location
- Scrap modelling table: Production.WorkOrder + Production.WorkOrderRouting +
↳Production.Product

Natural language request:

{user_question4}

Output requirements:

- Return valid JSON only.

""

```
[57]: message4 = [
    {
        "role": "system",
        "content": system
    },
    {
        "role": "user",
        "content": user_content7    # MUST be a string
    }
]
message4
```

```
[57]: [{'role': 'system',
        'content': 'You are an expert data analyst and SQL generator for the
AdventureWorks database.\n\nYour task is to translate a user's natural language
manufacturing or business question into a safe, correct SQL Server query and a
brief explanation.\n\nRules:\n1. Only generate read-only SQL.\n2. Allowed query
types are:\n    - SELECT\n    - Common Table Expressions (CTEs)\n    - ORDER BY\n    - GROUP BY\n    - HAVING\n    - JOINS\n    - window functions\n    - recursive CTEs
if needed\n3. Do not generate:\n    - INSERT\n    - UPDATE\n    - DELETE\n    -
DROP\n    - ALTER\n    - TRUNCATE\n    - EXEC\n    - MERGE\n    - stored
procedures\n4. The SQL dialect is Microsoft SQL Server (AdventureWorks).\n5.
Prefer fully qualified table names, for example:\n    Production.WorkOrder\n
Production.Product\n    Production.ProductInventory\n6. Use TOP for row limits,
not LIMIT.\n7. If the question is ambiguous, make the smallest reasonable
assumption and state it briefly.\n8. If the question cannot be answered from the
provided schema, say so clearly.\n9. Use joins only where supported by the
schema provided.\n10. Return output in valid JSON only.\n\nYou must return
```

```

exactly this JSON structure:\n\n{\n  "question_understanding": "short
restatement of the task",\n  "assumptions": ["assumption 1", "assumption 2"],\n
"sql": "single SQL Server query as a string",\n  "explanation": "brief
explanation of how the query works",\n  "result_shape": "what the output table
should contain",\n  "validation_checks": ["check 1", "check 2"],\n
"plot_suggestion": {\n    "recommended": true,\n    "plot_type":
"bar|line|pie|none",\n    "x": "column name or null",\n    "y": "column name or
null",\n    "title": "plot title or null"\n  }\n}\n\nAdditional requirements:\n-
The SQL must be executable as written.\n- Do not wrap SQL in markdown fences.\n-
If no chart is appropriate, set plot_suggestion.plot_type to "none" and other
plot fields to null.\n- If aggregation is used, ensure all non-aggregated
selected columns are included in GROUP BY.\n- If ranking is requested, prefer
ORDER BY and TOP, or window functions where appropriate.\n- If multi-level bill-
of-materials logic is requested, use a recursive CTE.'},
{'role': 'user',
 'content': '\nDatabase: AdventureWorks\nTask: Convert the following natural
language request into SQL Server SQL.\nSchema summary:\nProduction.WorkOrder\n-
WorkOrderID (PK)\n- ProductID (FK -> Production.Product.ProductID)\n-
OrderQty\n- StockedQty\n- ScrappedQty\n- StartDate\n- EndDate\n- DueDate\n-
ScrapReasonID\n\nProduction.Product\n- ProductID (PK)\n- Name\n-
ProductNumber\n- MakeFlag\n- FinishedGoodsFlag\n- Color\n- SafetyStockLevel\n-
ReorderPoint\n- StandardCost\n- ListPrice\n-
ProductSubcategoryID\n\nProduction.ProductInventory\n- ProductID (FK ->
Production.Product.ProductID)\n- LocationID (FK ->
Production.Location.LocationID)\n- Shelf\n- Bin\n-
Quantity\n\nProduction.Location\n- LocationID (PK)\n- Name\n- CostRate\n-
Availability\n\nProduction.WorkOrderRouting\n- WorkOrderID (FK ->
Production.WorkOrder.WorkOrderID)\n- ProductID\n- OperationSequence\n-
LocationID (FK -> Production.Location.LocationID)\n- ScheduledStartDate\n-
ScheduledEndDate\n- ActualStartDate\n- ActualEndDate\n- PlannedCost\n-
ActualCost\n\nProduction.BillOfMaterials\n- BillOfMaterialsID (PK)\n-
ProductAssemblyID\n- ComponentID\n- StartDate\n- EndDate\n- UnitMeasureCode\n-
BOMLevel\n- PerAssemblyQty\n\nProduction.TransactionHistory\n- TransactionID\n-
ProductID\n- ReferenceOrderID\n- ReferenceOrderLineID\n- TransactionDate\n-
TransactionType\n- Quantity\n- ActualCost\n\nExample:\n\nInput:\nCreate a work-
order-level modelling table for a scrap risk classifier.\nReturn one row per
WorkOrderID with:\n- scrap_flag\n- ProductID\n- product descriptor\n-
category/subcategory if available\n- OrderQty\n- operation_count\n-
distinct_locations\n\nOutput:\n{\n  "question_understanding": "Create a clean
work-order-level modelling table for predicting whether a work order will incur
any scrap.",\n  "assumptions": [\n    "The target is binary: scrap_flag = 1 if
ScrappedQty > 0, otherwise 0.",\n    "Each output row should represent one work
order.",\n    "The product descriptor is built from product name and product
number.",\n    "ProductSubcategoryID is used as the available category-related
feature." ]\n  },\n  "sql": "WITH WorkOrderBase AS ( SELECT wo.WorkOrderID, CASE
WHEN wo.ScrappedQty > 0 THEN 1 ELSE 0 END AS scrap_flag, wo.ProductID,
wo.OrderQty FROM Production.WorkOrder AS wo ), RoutingFeatures AS ( SELECT

```

```

wr.WorkOrderID, COUNT(*) AS operation_count, COUNT(DISTINCT wr.LocationID) AS
distinct_locations FROM Production.WorkOrderRouting AS wr GROUP BY
wr.WorkOrderID ), ProductFeatures AS ( SELECT p.ProductID, CONCAT(p.Name, \'
(\', p.ProductNumber, \')\') AS product_descriptor, p.ProductSubcategoryID FROM
Production.Product AS p ) SELECT wb.WorkOrderID, wb.scrap_flag, wb.ProductID,
pf.product_descriptor, pf.ProductSubcategoryID, wb.OrderQty, rf.operation_count,
rf.distinct_locations FROM WorkOrderBase AS wb JOIN ProductFeatures AS pf ON
wb.ProductID = pf.ProductID LEFT JOIN RoutingFeatures AS rf ON wb.WorkOrderID =
rf.WorkOrderID";\n "explanation": "The query builds a modelling table at work-
order level. It derives a binary scrap target from WorkOrder.ScrappedQty,
aggregates routing records to compute operation count and number of distinct
locations, and joins product details to provide candidate predictive
features.",\n "result_shape": "One row per WorkOrderID containing scrap_flag,
ProductID, product_descriptor, ProductSubcategoryID, OrderQty, operation_count,
and distinct_locations.",\n "validation_checks": [\n "Verify that each
WorkOrderID appears only once in the final output.",\n "Verify that
scrap_flag is 1 when ScrappedQty > 0 and 0 otherwise.",\n "Verify that
operation_count equals the number of WorkOrderRouting rows for each work
order.",\n "Verify that distinct_locations equals the number of unique
LocationID values per work order." ],\n "plot_suggestion": {\n
"recommended": false,\n "plot_type": "none",\n "x": null,\n "y":
null,\n "title": null\n }\n}\n\n---\n\nBusiness context:\n- This database
represents a manufacturing company.\n\nImportant table hints:\n- Work orders:
Production.WorkOrder + Production.Product\n- Inventory:
Production.ProductInventory + Production.Location + Production.Product\n-
Routing: Production.WorkOrderRouting + Production.Location\n- Scrap modelling
table: Production.WorkOrder + Production.WorkOrderRouting +
Production.Product\n\nNatural language request:\nScrap risk classifier
Question:\n"Can we predict whether a work order will incur any scrap?"
\nTarget:\n• Binary: scrap occurred (yes/no) on any routing row for that work
order\nFeatures:\n• Product, category\n• Locations visited\n• Operation count\n•
Quantity\n• Vendor mix of component parts (optional)\nModel:\n• Logistic
regression baseline + tree-based model\nMetrics:\n• ROC-AUC + precision/recall
(scrap is often imbalanced)\n\n\nOutput requirements:\n- Return valid JSON
only.\n'}]]

```

```

[58]: response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=message4 # correct
)
print(response.choices[0].message.content)

```

```

{
  "question_understanding": "Create a modelling table to predict whether a work
order will incur any scrap.",
  "assumptions": [
    "The target is binary: scrap_flag = 1 if any routing row for the work order
has ScrappedQty > 0, otherwise 0.",

```

```

    "Each output row should represent one work order.",
    "The product descriptor includes the product name and product number.",
    "ProductSubcategoryID is used as the category-related feature."
],
"sql": "WITH WorkOrderBase AS ( SELECT wo.WorkOrderID, CASE WHEN
SUM(wo.ScrapQty) > 0 THEN 1 ELSE 0 END AS scrap_flag, wo.ProductID,
wo.OrderQty FROM Production.WorkOrder AS wo GROUP BY wo.WorkOrderID,
wo.ProductID, wo.OrderQty ), RoutingFeatures AS ( SELECT wr.WorkOrderID,
COUNT(*) AS operation_count, COUNT(DISTINCT wr.LocationID) AS distinct_locations
FROM Production.WorkOrderRouting AS wr GROUP BY wr.WorkOrderID ),
ProductFeatures AS ( SELECT p.ProductID, CONCAT(p.Name, ' (', p.ProductNumber,
')') AS product_descriptor, p.ProductSubcategoryID FROM Production.Product AS p
) SELECT wb.WorkOrderID, wb.scrap_flag, wb.ProductID, pf.product_descriptor,
pf.ProductSubcategoryID, wb.OrderQty, rf.operation_count, rf.distinct_locations
FROM WorkOrderBase AS wb JOIN ProductFeatures AS pf ON wb.ProductID =
pf.ProductID LEFT JOIN RoutingFeatures AS rf ON wb.WorkOrderID =
rf.WorkOrderID;",
"explanation": "The query constructs a modelling table for work orders,
deriving a binary scrap target based on the sum of ScrapQty from the
WorkOrder table. It aggregates WorkOrderRouting records to gather the count of
operations and distinct locations, and joins product information to include
product descriptors and subcategories.",
"result_shape": "One row per WorkOrderID containing scrap_flag, ProductID,
product_descriptor, ProductSubcategoryID, OrderQty, operation_count, and
distinct_locations.",
"validation_checks": [
    "Verify that each WorkOrderID appears only once in the final output.",
    "Verify that scrap_flag is 1 when there is any scrap for the work order and
0 otherwise.",
    "Verify that operation_count corresponds to the number of WorkOrderRouting
entries for each work order.",
    "Verify that distinct_locations equals the number of unique LocationIDs from
routing for each work order."
],
"plot_suggestion": {
    "recommended": false,
    "plot_type": "none",
    "x": null,
    "y": null,
    "title": null
}
}

```

```

[59]: query8 = """
      WITH WorkOrderBase AS (
        SELECT
          wo.WorkOrderID,

```

```

        CASE
            WHEN wo.ScrapedQty > 0 THEN 1
            ELSE 0
        END AS scrap_flag,
        wo.ProductID,
        wo.OrderQty
    FROM Production.WorkOrder AS wo
),
RoutingFeatures AS (
    SELECT
        wr.WorkOrderID,
        COUNT(*) AS operation_count,
        COUNT(DISTINCT wr.LocationID) AS distinct_locations
    FROM Production.WorkOrderRouting AS wr
    GROUP BY wr.WorkOrderID
),
ProductFeatures AS (
    SELECT
        p.ProductID,
        CONCAT(p.Name, ' (' , p.ProductNumber, ')') AS product_descriptor,
        p.ProductSubcategoryID
    FROM Production.Product AS p
)
SELECT
    wb.WorkOrderID,
    wb.scrap_flag,
    wb.ProductID,
    pf.product_descriptor,
    pf.ProductSubcategoryID,
    wb.OrderQty,
    rf.operation_count,
    rf.distinct_locations
FROM WorkOrderBase AS wb
JOIN ProductFeatures AS pf
    ON wb.ProductID = pf.ProductID
LEFT JOIN RoutingFeatures AS rf
    ON wb.WorkOrderID = rf.WorkOrderID;
"""

table8 = pd.read_sql(query8, conn)
table8

```

C:\Users\emper\AppData\Local\Temp\ipykernel_55500\1544695553.py:44: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
table8 = pd.read_sql(query8, conn)
```



```
[59]:
```

	WorkOrderID	scrap_flag	ProductID \
0	1	0	722
1	2	0	725
2	3	0	726
3	4	0	729
4	5	0	730
...	...	...	...
72586	72587	0	804
72587	72588	0	316
72588	72589	0	331
72589	72590	0	350
72590	72591	0	531

	product_descriptor	ProductSubcategoryID	OrderQty \
0	LL Road Frame - Black, 58 (FR-R38B-58)	14.0	8
1	LL Road Frame - Red, 44 (FR-R38R-44)	14.0	15
2	LL Road Frame - Red, 48 (FR-R38R-48)	14.0	9
3	LL Road Frame - Red, 60 (FR-R38R-60)	14.0	16
4	LL Road Frame - Red, 62 (FR-R38R-62)	14.0	14
...	...	...	...
72586	HL Fork (FK-9939)	10.0	19
72587	Blade (BL-2036)	NaN	52
72588	Fork End (FE-3760)	NaN	52
72589	Fork Crown (FC-3654)	NaN	26
72590	Steerer (SR-2098)	NaN	26

	operation_count	distinct_locations
0	NaN	NaN
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN
4	NaN	NaN
...	...	...
72586	2.0	2.0
72587	NaN	NaN
72588	NaN	NaN
72589	NaN	NaN
72590	NaN	NaN


```
[72591 rows x 8 columns]
```

```
[60]: #table8.to_csv("scrap2_df.csv", index=False)
```

I realised that the output generated from the prompt for question 4 was just a table needed to build the models started in the assignment brief instead of a qualitative response that answers whether or not we can predict a work order will incur any scrap. I then send the following prompt to ChatGPT:

Since the assignment is about vibe coding using Natural Language, why can't I just use a prompt to generate the answer to the question from the NL instead of building the models, and got this response:

Short answer: you can use NL prompts—but not instead of building the model for this assignment. What you're proposing is a pure LLM answer (reasoning without data). What the assignment expects is a data-driven analytical pipeline (NL → SQL → data → model → insight).

ChatGPT explained further that the prompting by vibe coding alone is not enough because the problem requires predictive analysis, not descriptive. What this means is that the question require ChatGPT to learn pattern from the historical data in the database, estimate the probabilities of each work order incurring scrap or not, group them into the binary scrap class and then carry out performance evaluation of the predictive power of the model by calculating their accuracy, precision, recall, F-score, roc_auc_score, confusion matrix, classification_report, and then plot ROC curve.

ChatGPT concludes that **“An LLM alone cannot learn from your database unless you explicitly feed it the data.”**. This conclusion outlines the limit of the reasoning ability of the LLM in its current state and by extension the limit of using vibe coding via NL-SQL interface to solve quantitative manufacturing problems that require more than just expressing opinions. If the LLM must answer such a question in its current state, it will have to perform **“hallucinated analytics”** that will requires it guessing the pattern instead of using actual historical data. This approach will be methodologically incorrect.

ChatGPT suggests that the LLM in its current state can only be used to perform the following tasks in relation to the question asked:

- Generate the SQL query for the required table
- Determine the predictive feature for whether an order will incur scrap or not (in this case operation_count)
- Recommend what the company can do to reduce scrap

Now, let's but the models using the generated table

```
[62]: scrap_df = pd.read_csv("scrap2_df.csv")
      scrap_df.head()
```

```
[62]:  WorkOrderID  scrap_flag  ProductID  product_descriptor \
0           1           0         722  LL Road Frame - Black, 58 (FR-R38B-58)
1           2           0         725   LL Road Frame - Red, 44 (FR-R38R-44)
2           3           0         726   LL Road Frame - Red, 48 (FR-R38R-48)
3           4           0         729   LL Road Frame - Red, 60 (FR-R38R-60)
4           5           0         730   LL Road Frame - Red, 62 (FR-R38R-62)
```

```
      ProductSubcategoryID  OrderQty  operation_count  distinct_locations
0                14.0           8           NaN           NaN
1                14.0          15           NaN           NaN
2                14.0           9           NaN           NaN
3                14.0          16           NaN           NaN
4                14.0          14           NaN           NaN
```

```
[63]: scrap_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72591 entries, 0 to 72590
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   WorkOrderID            72591 non-null  int64
1   scrap_flag             72591 non-null  int64
2   ProductID              72591 non-null  int64
3   product_descriptor     72591 non-null  object
4   ProductSubcategoryID   49541 non-null  float64
5   OrderQty               72591 non-null  int64
6   operation_count        42625 non-null  float64
7   distinct_locations     42625 non-null  float64
dtypes: float64(3), int64(4), object(1)
memory usage: 4.4+ MB
```

WorkOrderID, ProductID, and ProductSubcategoryID were assigned wrong data. They are categorical data wrongly labeled as numerical data (integer). Let's change them to the right data type

```
[65]: scrap_df["WorkOrderID"]=scrap_df["WorkOrderID"].astype("object")
```

```
[66]: scrap_df["ProductID"]=scrap_df["ProductID"].astype("object")
```

```
[67]: scrap_df["ProductSubcategoryID"]=scrap_df["ProductSubcategoryID"].
      ↪astype("object")
```

```
[68]: scrap_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72591 entries, 0 to 72590
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   WorkOrderID            72591 non-null  object
1   scrap_flag             72591 non-null  int64
2   ProductID              72591 non-null  object
3   product_descriptor     72591 non-null  object
4   ProductSubcategoryID   49541 non-null  object
5   OrderQty               72591 non-null  int64
6   operation_count        42625 non-null  float64
7   distinct_locations     42625 non-null  float64
dtypes: float64(2), int64(2), object(4)
memory usage: 4.4+ MB
```

```
[69]: # Let's check the missing values
      scrap_df.isna().sum()
```

```
[69]: WorkOrderID          0
      scrap_flag          0
      ProductID           0
      product_descriptor   0
      ProductSubcategoryID 23050
      OrderQty            0
      operation_count      29966
      distinct_locations   29966
      dtype: int64
```

Because the dataset is large (72591), I will just delete the missing column to avoid introducing unnecessary bias associated with imputation

```
[71]: scrap_df= scrap_df.dropna().reset_index()
      scrap_df.isna().sum()
```

```
[71]: index          0
      WorkOrderID    0
      scrap_flag     0
      ProductID      0
      product_descriptor 0
      ProductSubcategoryID 0
      OrderQty       0
      operation_count 0
      distinct_locations 0
      dtype: int64
```

```
[72]: scrap_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 38079 entries, 0 to 38078
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   index                 38079 non-null  int64
1   WorkOrderID           38079 non-null  object
2   scrap_flag            38079 non-null  int64
3   ProductID             38079 non-null  object
4   product_descriptor    38079 non-null  object
5   ProductSubcategoryID  38079 non-null  object
6   OrderQty              38079 non-null  int64
7   operation_count        38079 non-null  float64
8   distinct_locations    38079 non-null  float64
dtypes: float64(2), int64(3), object(4)
memory usage: 2.6+ MB
```

6.2 Feature selection

```
[74]: X = scrap_df.drop(columns = ["scrap_flag"])
      X.head()
```

```
[74]:   index WorkOrderID ProductID      product_descriptor \
0      12          13      747  HL Mountain Frame - Black, 38 (FR-M94B-38)
1      13          14      748  HL Mountain Frame - Silver, 38 (FR-M94S-38)
2      14          15      749      Road-150 Red, 62 (BK-R93R-62)
3      15          16      753      Road-150 Red, 56 (BK-R93R-56)
4      16          17      754      Road-450 Red, 58 (BK-R68R-58)

      ProductSubcategoryID  OrderQty  operation_count  distinct_locations
0              12.0          4              6.0              6.0
1              12.0          2              6.0              6.0
2               2.0          4              1.0              1.0
3               2.0         14              1.0              1.0
4               2.0         27              1.0              1.0
```

```
[75]: y = scrap_df["scrap_flag"]
      y.head()
```

```
[75]: 0    0
      1    0
      2    0
      3    0
      4    0
      Name: scrap_flag, dtype: int64
```

```
[76]: y.value_counts()
```

```
[76]: scrap_flag
0     37854
1       225
      Name: count, dtype: int64
```

There is a large class imbalance in favour of the no scrap class. We shall use stratified sampling and `class_weight = "balance"` parameter to cater for this

Let's drop `product_descriptor` and `WorkOrderID`

```
[79]: X = X.drop(columns = ["product_descriptor", "WorkOrderID"])
      X.head()
```

```
[79]:   index ProductID ProductSubcategoryID  OrderQty  operation_count \
0      12        747              12.0          4              6.0
1      13        748              12.0          2              6.0
2      14        749               2.0          4              1.0
3      15        753               2.0         14              1.0
```

4	16	754	2.0	27	1.0
---	----	-----	-----	----	-----

	distinct_locations
0	6.0
1	6.0
2	1.0
3	1.0
4	1.0

```
[80]: X.ProductSubcategoryID.value_counts()
```

```
[80]: ProductSubcategoryID
17.0    8201
2.0     6623
4.0     4433
1.0     3864
5.0     2672
10.0    2539
9.0     2207
3.0     2031
11.0    2011
8.0     2008
12.0    1225
14.0     265
Name: count, dtype: int64
```

```
[81]: X = pd.get_dummies(
      X,
      columns=["ProductSubcategoryID"],
      drop_first=True
    )

X.head()
```

```
[81]:
```

	index	ProductID	OrderQty	operation_count	distinct_locations	\
0	12	747	4	6.0	6.0	
1	13	748	2	6.0	6.0	
2	14	749	4	1.0	1.0	
3	15	753	14	1.0	1.0	
4	16	754	27	1.0	1.0	

	ProductSubcategoryID_2.0	ProductSubcategoryID_3.0	\
0	False	False	
1	False	False	
2	True	False	
3	True	False	
4	True	False	

	ProductSubcategoryID_4.0	ProductSubcategoryID_5.0 \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

	ProductSubcategoryID_8.0	ProductSubcategoryID_9.0 \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

	ProductSubcategoryID_10.0	ProductSubcategoryID_11.0 \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

	ProductSubcategoryID_12.0	ProductSubcategoryID_14.0 \
0	True	False
1	True	False
2	False	False
3	False	False
4	False	False

	ProductSubcategoryID_17.0
0	False
1	False
2	False
3	False
4	False

```
[82]: X = X.drop(columns = ["index", "ProductID"])
      X.head()
```

	OrderQty	operation_count	distinct_locations	ProductSubcategoryID_2.0 \
0	4	6.0	6.0	False
1	2	6.0	6.0	False
2	4	1.0	1.0	True
3	14	1.0	1.0	True
4	27	1.0	1.0	True

	ProductSubcategoryID_3.0	ProductSubcategoryID_4.0 \
--	--------------------------	----------------------------

0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

	ProductSubcategoryID_5.0	ProductSubcategoryID_8.0 \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

	ProductSubcategoryID_9.0	ProductSubcategoryID_10.0 \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

	ProductSubcategoryID_11.0	ProductSubcategoryID_12.0 \
0	False	True
1	False	True
2	False	False
3	False	False
4	False	False

	ProductSubcategoryID_14.0	ProductSubcategoryID_17.0
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

```
[83]: # Let's convert the dummies to integers
X = X.astype(int)
X.head()
```

	OrderQty	operation_count	distinct_locations	ProductSubcategoryID_2.0 \
0	4	6	6	0
1	2	6	6	0
2	4	1	1	1
3	14	1	1	1
4	27	1	1	1

	ProductSubcategoryID_3.0	ProductSubcategoryID_4.0 \
0	0	0

1	0	0
2	0	0
3	0	0
4	0	0

	ProductSubcategoryID_5.0	ProductSubcategoryID_8.0 \
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0

	ProductSubcategoryID_9.0	ProductSubcategoryID_10.0 \
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0

	ProductSubcategoryID_11.0	ProductSubcategoryID_12.0 \
0	0	1
1	0	1
2	0	0
3	0	0
4	0	0

	ProductSubcategoryID_14.0	ProductSubcategoryID_17.0
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0

```
[84]: # Let's split the data into test-train
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
[85]: from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(max_iter=1000, class_weight = "balanced")
lr.fit(X_train, y_train)
```

```
[85]: LogisticRegression(class_weight='balanced', max_iter=1000)
```

```
[86]: from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, class_weight = "balanced",
    random_state=42)
rf.fit(X_train, y_train)
```

```
[86]: RandomForestClassifier(class_weight='balanced', random_state=42)
```

We used the `class_weight = "balance"` to cater for the large class imbalance in the outcome variable.

```
[88]: #Let's introduce the test data to predict outcome for y using both models
# Logistic
y_pred_lr = lr.predict(X_test)
y_prob_lr = lr.predict_proba(X_test)[:, 1]

# Random Forest
y_pred_rf = rf.predict(X_test)
y_prob_rf = rf.predict_proba(X_test)[:, 1]
```

```
[89]: from sklearn.metrics import confusion_matrix

# Logistic Regression Confusion Matrix
cm_lr = confusion_matrix(y_test, y_pred_lr)
#print("\nLogistic Confusion Matrix:\n", cm_lr)

# Random Forest Confusion Matrix
cm_rf = confusion_matrix(y_test, y_pred_rf)
#print("\nRandom Forest Confusion Matrix:\n", cm_rf)

cm_lr_df = pd.DataFrame(cm_lr,
                        index=["Actual 0", "Actual 1"],
                        columns=["Predicted 0", "Predicted 1"])

cm_rf_df = pd.DataFrame(cm_rf,
                        index=["Actual 0", "Actual 1"],
                        columns=["Predicted 0", "Predicted 1"])

print("\nLogistic Confusion Matrix:\n", cm_lr_df)
print("\nRandom Forest Confusion Matrix:\n", cm_rf_df)
```

Logistic Confusion Matrix:

	Predicted 0	Predicted 1
Actual 0	7123	448
Actual 1	2	43

Random Forest Confusion Matrix:

	Predicted 0	Predicted 1
--	-------------	-------------

Actual 0	7473	98
Actual 1	34	11

```
[90]: from sklearn.metrics import roc_auc_score, classification_report

print("Logistic ROC-AUC:", roc_auc_score(y_test, y_prob_lr))
print("RF ROC-AUC:", roc_auc_score(y_test, y_prob_rf))

print("\nLogistic Report:\n", classification_report(y_test, y_pred_lr))
print("\nRF Report:\n", classification_report(y_test, y_pred_rf))
```

Logistic ROC-AUC: 0.948737140257415
 RF ROC-AUC: 0.687152731915643

Logistic Report:

	precision	recall	f1-score	support
0	1.00	0.94	0.97	7571
1	0.09	0.96	0.16	45
accuracy			0.94	7616
macro avg	0.54	0.95	0.56	7616
weighted avg	0.99	0.94	0.96	7616

RF Report:

	precision	recall	f1-score	support
0	1.00	0.99	0.99	7571
1	0.10	0.24	0.14	45
accuracy			0.98	7616
macro avg	0.55	0.62	0.57	7616
weighted avg	0.99	0.98	0.99	7616

```
[249]: # -----
# Performance metrics from your results
# -----
metrics = pd.DataFrame({
    "Metric": ["ROC-AUC", "Accuracy", "Precision (Class 1)", "Recall (Class_
↵1)", "F1-score (Class 1)"],
    "Logistic Regression": [0.9487, 0.94, 0.09, 0.96, 0.16],
    "Random Forest":      [0.6872, 0.98, 0.10, 0.24, 0.14]
})

# -----
# Side-by-side bar chart
```

```

# -----
x = np.arange(len(metrics["Metric"]))
width = 0.38

plt.figure(figsize=(14,7))

plt.bar(
    x - width/2,
    metrics["Logistic Regression"],
    width,
    label="Logistic Regression"
)

plt.bar(
    x + width/2,
    metrics["Random Forest"],
    width,
    label="Random Forest"
)

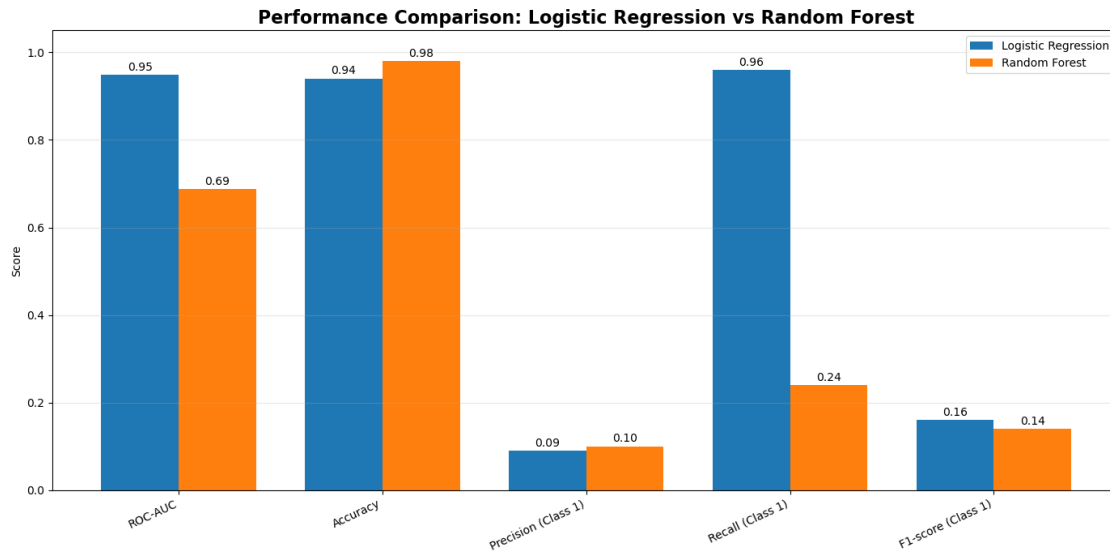
# Labels
plt.xticks(x, metrics["Metric"], rotation=25, ha="right")
plt.ylabel("Score")
plt.title("Performance Comparison: Logistic Regression vs Random Forest",
    ↪fontsize=16, weight="bold")
plt.ylim(0, 1.05)
plt.legend()

# Value labels
for i, val in enumerate(metrics["Logistic Regression"]):
    plt.text(i - width/2, val + 0.01, f"{val:.2f}", ha="center", fontsize=10)

for i, val in enumerate(metrics["Random Forest"]):
    plt.text(i + width/2, val + 0.01, f"{val:.2f}", ha="center", fontsize=10)

plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

```



```
[91]: # Let's examine the contribution of each factor
```

```
[92]: # Estimate the coefficients
```

```
coeffs = pd.Series(lr.coef_[0], index=X.columns).sort_values()
coeffs
```

```
[92]: ProductSubcategoryID_9.0    -17.481131
      ProductSubcategoryID_10.0   -5.611196
      ProductSubcategoryID_5.0    -2.324267
      ProductSubcategoryID_11.0   -1.529892
      ProductSubcategoryID_17.0   -1.424766
      ProductSubcategoryID_8.0    -1.180661
      ProductSubcategoryID_4.0    -1.163476
      operation_count             -0.116381
      distinct_locations          -0.116381
      OrderQty                    0.018484
      ProductSubcategoryID_3.0    0.049378
      ProductSubcategoryID_2.0    0.149405
      ProductSubcategoryID_12.0   0.962727
      ProductSubcategoryID_14.0   2.456226
      dtype: float64
```

```
[93]: importances = pd.Series(rf.feature_importances_, index=X.columns)
      importances.sort_values(ascending=False)
```

```
[93]: OrderQty                    0.967979
      ProductSubcategoryID_9.0    0.008609
      ProductSubcategoryID_2.0    0.005450
      operation_count             0.003934
```

distinct_locations	0.002809
ProductSubcategoryID_14.0	0.002500
ProductSubcategoryID_17.0	0.001397
ProductSubcategoryID_11.0	0.001388
ProductSubcategoryID_10.0	0.001317
ProductSubcategoryID_5.0	0.001194
ProductSubcategoryID_3.0	0.001144
ProductSubcategoryID_8.0	0.000973
ProductSubcategoryID_12.0	0.000814
ProductSubcategoryID_4.0	0.000493

dtype: float64

7 Now let's build the NL interpretation layer

```
[95]: # Metrics
roc_auc_lr = roc_auc_score(y_test, y_prob_lr)
roc_auc_rf = roc_auc_score(y_test, y_prob_rf)

report_lr = classification_report(y_test, y_pred_lr, output_dict=True,
    ↪zero_division=0)
report_rf = classification_report(y_test, y_pred_rf, output_dict=True,
    ↪zero_division=0)

cm_lr = confusion_matrix(y_test, y_pred_lr)
cm_rf = confusion_matrix(y_test, y_pred_rf)

# Logistic coefficients
coef_lr = pd.Series(lr.coef_[0], index=X_train.columns).
    ↪sort_values(ascending=False)

top_positive_lr = coef_lr.head(10).to_dict()
top_negative_lr = coef_lr.tail(10).to_dict()

# Random forest feature importance
feat_rf = pd.Series(rf.feature_importances_, index=X_train.columns).
    ↪sort_values(ascending=False)
top_features_rf = feat_rf.head(10).to_dict()
```

```
[96]: interpretation_user_content = f"""
You are a manufacturing data science analyst.

Interpret the results of a scrap risk classification task for AdventureWorks.

Context:
- Target: scrap_flag (1 = work order incurred scrap, 0 = no scrap)
- Models compared: Logistic Regression and Random Forest
```

- Features include order quantity, operation count, distinct locations, and one-hot encoded product subcategory fields.

Model performance:

- Logistic Regression ROC-AUC: {roc_auc_lr:.4f}
- Random Forest ROC-AUC: {roc_auc_rf:.4f}

Logistic Regression classification report:

```
{pd.DataFrame(report_lr).transpose().to_string()}
```

Random Forest classification report:

```
{pd.DataFrame(report_rf).transpose().to_string()}
```

Logistic Regression confusion matrix:

```
{cm_lr}
```

Random Forest confusion matrix:

```
{cm_rf}
```

Top positive Logistic Regression coefficients:

```
{top_positive_lr}
```

Top negative Logistic Regression coefficients:

```
{top_negative_lr}
```

Top Random Forest feature importances:

```
{top_features_rf}
```

Return valid JSON only using this structure:

```
{{
  "overall_assessment": "short summary of model performance",
  "best_model": "which model performed better and why",
  "top_drivers": [
    "driver 1",
    "driver 2",
    "driver 3"
  ],
  "shop_floor_recommendations": [
    "recommendation 1",
    "recommendation 2",
    "recommendation 3"
  ],
  "limitations": [
    "limitation 1",
    "limitation 2"
  ]
}}
```

```
"""
```

```
[97]: message4 = [
    {
        "role": "system",
        "content": "You are an expert manufacturing data analyst. Be
↪evidence-based and concise. Return valid JSON only."
    },
    {
        "role": "user",
        "content": interpretation_user_content    # MUST be a string
    }
]
message4
```

```
[97]: [{'role': 'system',
        'content': 'You are an expert manufacturing data analyst. Be evidence-based
and concise. Return valid JSON only.'},
        {'role': 'user',
        'content': '\nYou are a manufacturing data science analyst.\n\nInterpret the
results of a scrap risk classification task for AdventureWorks.\n\nContext:\n-
Target: scrap_flag (1 = work order incurred scrap, 0 = no scrap)\n- Models
compared: Logistic Regression and Random Forest\n- Features include order
quantity, operation count, distinct locations, and one-hot encoded product
subcategory fields.\n\nModel performance:\n- Logistic Regression ROC-AUC:
0.9487\n- Random Forest ROC-AUC: 0.6872\n\nLogistic Regression classification
report:\n
                precision    recall  f1-score   support\n
0.999719  0.940827  0.969379  7571.000000\n1                0.087576  0.955556
0.160448   45.000000\naccuracy                0.940914  0.940914  0.940914
0.940914\nmacro avg                0.543648  0.948191  0.564914  7616.000000\nweighted avg
0.994330  0.940914  0.964600  7616.000000\n\nRandom Forest classification
report:\n
                precision    recall  f1-score   support\n
0.995471  0.987056  0.991246  7571.000000\n1                0.100917  0.244444
0.142857   45.000000\naccuracy                0.982668  0.982668  0.982668
0.982668\nmacro avg                0.548194  0.615750  0.567051  7616.000000\nweighted avg
0.990185  0.982668  0.986233  7616.000000\n\nLogistic Regression confusion
matrix:\n[[7123  448]\n [   2   43]]\n\nRandom Forest confusion matrix:\n[[7473
 98]\n [  34   11]]\n\nTop positive Logistic Regression
coefficients:\n{'ProductSubcategoryID_14.0': 2.4562262577133795,
'\ProductSubcategoryID_12.0': 0.9627268603303911, '\ProductSubcategoryID_2.0':
0.1494054996220442, '\ProductSubcategoryID_3.0': 0.04937832495716074,
'\OrderQty': 0.0184839843694212, '\operation_count': -0.11638075711724685,
'\distinct_locations': -0.11638075711724685, '\ProductSubcategoryID_4.0':
-1.16347641057093, '\ProductSubcategoryID_8.0': -1.1806608040839166,
'\ProductSubcategoryID_17.0': -1.4247663817193492}\n\nTop negative Logistic
Regression coefficients:\n{'OrderQty': 0.0184839843694212,
'\operation_count': -0.11638075711724685, '\distinct_locations':
```



```
-0.11638075711724685, \'ProductSubcategoryID_4.0\': -1.16347641057093,
\'ProductSubcategoryID_8.0\': -1.1806608040839166,
\'ProductSubcategoryID_17.0\': -1.4247663817193492,
\'ProductSubcategoryID_11.0\': -1.5298917956364448,
\'ProductSubcategoryID_5.0\': -2.3242672844584127,
\'ProductSubcategoryID_10.0\': -5.611196143899333, \'ProductSubcategoryID_9.0\':
-17.481131433180437}\n\nTop Random Forest feature importances:\n{\n\'OrderQty\':
0.9679788081244174, \'ProductSubcategoryID_9.0\': 0.00860876323256369,
\'ProductSubcategoryID_2.0\': 0.005449503486976633, \'operation_count\':
0.003934016103342862, \'distinct_locations\': 0.002808529918772683,
\'ProductSubcategoryID_14.0\': 0.0025003141551194555,
\'ProductSubcategoryID_17.0\': 0.0013966479543035583,
\'ProductSubcategoryID_11.0\': 0.0013878418844122864,
\'ProductSubcategoryID_10.0\': 0.0013173670140373692,
\'ProductSubcategoryID_5.0\': 0.0011936391611184996}\n\nReturn valid JSON only
using this structure:\n{\n  "overall_assessment": "short summary of model
performance",\n  "best_model": "which model performed better and why",\n
  "top_drivers": [\n    "driver 1",\n    "driver 2",\n    "driver 3"\n  ],\n
  "shop_floor_recommendations": [\n    "recommendation 1",\n    "recommendation
2",\n    "recommendation 3"\n  ],\n  "limitations": [\n    "limitation 1",\n
  "limitation 2"\n  ]\n}
```

```
[98]: response_interpret = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=message4
    )
```

```
[99]: # Let's parse the response_interpret
import json

raw_interpret = response_interpret.choices[0].message.content.strip()

if raw_interpret.startswith("`json`"):
    raw_interpret = raw_interpret.removeprefix("`json`").removesuffix("`").
    ↪strip()
elif raw_interpret.startswith("`"):
    raw_interpret = raw_interpret.removeprefix("`").removesuffix("`").
    ↪strip()

interpretation_plan = json.loads(raw_interpret)
interpretation_plan
```

```
[99]: {'overall_assessment': 'The Logistic Regression model outperformed the Random
Forest model significantly, highlighting its capability to differentiate between
scrap and non-scrap work orders.',
       'best_model': 'Logistic Regression performed better due to its higher ROC-AUC
score of 0.9487 compared to Random Forest's 0.6872, as well as superior
```

```
precision and recall for detecting scrap instances.",
'top_drivers': ['ProductSubcategoryID_14.0',
'OrderQty',
'ProductSubcategoryID_9.0'],
'shop_floor_recommendations': ['Focus on products in subcategory 14 to minimize
scrap.',
'Monitor order quantities to manage scrap risks effectively.',
'Train staff on processes related to high-scrap subcategories.'],
'limitations': ['Logistic Regression has limited ability to capture complex
interactions compared to Random Forest.',
'The model may be biased due to imbalanced classes, suggesting further
exploration into underrepresented scrap instances.']]}
```

```
[100]: print("Overall assessment:", interpretation_plan["overall_assessment"])
print("Best model:", interpretation_plan["best_model"])

print("\nTop drivers:")
for item in interpretation_plan["top_drivers"]:
    print("-", item)

print("\nShop-floor recommendations:")
for item in interpretation_plan["shop_floor_recommendations"]:
    print("-", item)

print("\nLimitations:")
for item in interpretation_plan["limitations"]:
    print("-", item)
```

Overall assessment: The Logistic Regression model outperformed the Random Forest model significantly, highlighting its capability to differentiate between scrap and non-scrap work orders.

Best model: Logistic Regression performed better due to its higher ROC-AUC score of 0.9487 compared to Random Forest's 0.6872, as well as superior precision and recall for detecting scrap instances.

Top drivers:

- ProductSubcategoryID_14.0
- OrderQty
- ProductSubcategoryID_9.0

Shop-floor recommendations:

- Focus on products in subcategory 14 to minimize scrap.
- Monitor order quantities to manage scrap risks effectively.
- Train staff on processes related to high-scrap subcategories.

Limitations:

- Logistic Regression has limited ability to capture complex interactions compared to Random Forest.

- The model may be biased due to imbalanced classes, suggesting further exploration into underrepresented scrap instances.

8 Question 5

```
[102]: user_question5 = """
Forecast which finished goods will be at risk of stockout given recent_
↳ transaction history and current on-hand. Approach:
• Build consumption signals from TransactionHistory (sales/work order issues)
• Simple baseline: moving-average demand + reorder-point logic
• ML: classification of "stockout next N days" using lag features
"""
```

```
[103]: user_content8 = f"""
Database: AdventureWorks
Task: Convert the following natural language request into SQL Server SQL.

Schema summary:
{database_schema}

Business context:
- This database represents a manufacturing company.
- The goal is to support inventory risk analysis for finished goods.

Important table hints:
- Inventory: Production.ProductInventory + Production.Product + Production.
  ↳ Location
- Transactions: Production.TransactionHistory + Production.Product
- Stockout modelling table: Production.ProductInventory + Production.
  ↳ TransactionHistory + Production.Product

Natural language request:
{user_question5}

Output requirements:
- Return valid JSON only.
"""
```

```
[104]: message5 = [
    {
        "role": "system",
        "content": system
    },
    {
        "role": "user",
        "content": user_content8    # MUST be a string
    }
]
```

```

    }
]
message5

```

```

[104]: [{ 'role': 'system',
  'content': 'You are an expert data analyst and SQL generator for the
AdventureWorks database.\n\nYour task is to translate a user's natural language
manufacturing or business question into a safe, correct SQL Server query and a
brief explanation.\n\nRules:\n1. Only generate read-only SQL.\n2. Allowed query
types are:\n  - SELECT\n  - Common Table Expressions (CTEs)\n  - ORDER BY\n  - GROUP BY\n  - HAVING\n  - JOINS\n  - window functions\n  - recursive CTEs
if needed\n3. Do not generate:\n  - INSERT\n  - UPDATE\n  - DELETE\n  -
DROP\n  - ALTER\n  - TRUNCATE\n  - EXEC\n  - MERGE\n  - stored
procedures\n4. The SQL dialect is Microsoft SQL Server (AdventureWorks).\n5.
Prefer fully qualified table names, for example:\n  Production.WorkOrder\n
Production.Product\n  Production.ProductInventory\n6. Use TOP for row limits,
not LIMIT.\n7. If the question is ambiguous, make the smallest reasonable
assumption and state it briefly.\n8. If the question cannot be answered from the
provided schema, say so clearly.\n9. Use joins only where supported by the
schema provided.\n10. Return output in valid JSON only.\n\nYou must return
exactly this JSON structure:\n\n{\n  "question_understanding": "short
restatement of the task",\n  "assumptions": ["assumption 1", "assumption 2"],\n
"sql": "single SQL Server query as a string",\n  "explanation": "brief
explanation of how the query works",\n  "result_shape": "what the output table
should contain",\n  "validation_checks": ["check 1", "check 2"],\n
"plot_suggestion": {\n    "recommended": true,\n    "plot_type":
"bar|line|pie|none",\n    "x": "column name or null",\n    "y": "column name or
null",\n    "title": "plot title or null"\n  }\n}\n\nAdditional requirements:\n-
The SQL must be executable as written.\n- Do not wrap SQL in markdown fences.\n-
If no chart is appropriate, set plot_suggestion.plot_type to "none" and other
plot fields to null.\n- If aggregation is used, ensure all non-aggregated
selected columns are included in GROUP BY.\n- If ranking is requested, prefer
ORDER BY and TOP, or window functions where appropriate.\n- If multi-level bill-
of-materials logic is requested, use a recursive CTE.'},
{ 'role': 'user',
  'content': '\nDatabase: AdventureWorks\nTask: Convert the following natural
language request into SQL Server SQL.\n\nSchema
summary:\nProduction.WorkOrder\n- WorkOrderID (PK)\n- ProductID (FK ->
Production.Product.ProductID)\n- OrderQty\n- StockedQty\n- ScrappedQty\n-
StartDate\n- EndDate\n- DueDate\n- ScrapReasonID\n\nProduction.Product\n-
ProductID (PK)\n- Name\n- ProductNumber\n- MakeFlag\n- FinishedGoodsFlag\n-
Color\n- SafetyStockLevel\n- ReorderPoint\n- StandardCost\n- ListPrice\n-
ProductSubcategoryID\n\nProduction.ProductInventory\n- ProductID (FK ->
Production.Product.ProductID)\n- LocationID (FK ->
Production.Location.LocationID)\n- Shelf\n- Bin\n-
Quantity\n\nProduction.Location\n- LocationID (PK)\n- Name\n- CostRate\n-
Availability\n\nProduction.WorkOrderRouting\n- WorkOrderID (FK ->

```

Production.WorkOrder.WorkOrderID)\n- ProductID\n- OperationSequence\n- LocationID (FK -> Production.Location.LocationID)\n- ScheduledStartDate\n- ScheduledEndDate\n- ActualStartDate\n- ActualEndDate\n- PlannedCost\n- ActualCost\n\nProduction.BillOfMaterials\n- BillOfMaterialsID (PK)\n- ProductAssemblyID\n- ComponentID\n- StartDate\n- EndDate\n- UnitMeasureCode\n- BOMLevel\n- PerAssemblyQty\n\nProduction.TransactionHistory\n- TransactionID\n- ProductID\n- ReferenceOrderID\n- ReferenceOrderLineID\n- TransactionDate\n- TransactionType\n- Quantity\n- ActualCost\n\nBusiness context:\n- This database represents a manufacturing company.\n- The goal is to support inventory risk analysis for finished goods.\n\nImportant table hints:\n- Inventory: Production.ProductInventory + Production.Product + Production.Location\n- Transactions: Production.TransactionHistory + Production.Product\n- Stockout modelling table: Production.ProductInventory + Production.TransactionHistory + Production.Product\n\nNatural language request:\n\nForecast which finished goods will be at risk of stockout given recent transaction history and current on-hand. Approach:\n• Build consumption signals from TransactionHistory (sales/work order issues)\n• Simple baseline: moving-average demand + reorder-point logic\n• ML: classification of "stockout next N days" using lag features\n\nOutput requirements:\n- Return valid JSON only.\n']}]

```
[105]: response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=message5 # correct
)
print(response.choices[0].message.content)
```

```
{
  "question_understanding": "Analyze finished goods inventory risk of stockouts based on transaction history and current inventory levels.",
  "assumptions": ["Only finished goods are considered for forecasting stockouts.", "Moving-average demand can't be calculated directly due to lack of time period specifics."],
  "sql": "WITH RecentTransactions AS ( SELECT ProductID, SUM(Quantity) AS TotalSold FROM Production.TransactionHistory WHERE TransactionDate >= DATEADD(DAY, -30, GETDATE()) GROUP BY ProductID ), CurrentInventory AS ( SELECT p.ProductID, p.Name, pi.Quantity, p.ReorderPoint FROM Production.Product AS p JOIN Production.ProductInventory AS pi ON p.ProductID = pi.ProductID WHERE p.FinishedGoodsFlag = 1 ) SELECT c.ProductID, c.Name, c.Quantity, c.ReorderPoint, r.TotalSold, CASE WHEN c.Quantity <= c.ReorderPoint THEN 'At Risk' ELSE 'Sufficient' END AS StockRisk FROM CurrentInventory AS c LEFT JOIN RecentTransactions AS r ON c.ProductID = r.ProductID;",
  "explanation": "The query creates two Common Table Expressions (CTEs): the first calculates the total quantity sold from the last 30 days for each product, and the second retrieves current inventory levels for finished goods along with their reorder points. Then, it combines this data to assess if the current inventory is at risk of stockout, marking each product as 'At Risk' or 'Sufficient'.",
  "result_shape": "The output table contains columns for ProductID, Name,
```

```

Quantity on hand, Reorder Point, Total Quantity Sold in the last 30 days, and a
status indicating stock risk.",
  "validation_checks": ["Ensure that only finished goods are included in the
results.", "Confirm that stock risk is determined correctly based on inventory
compared to reorder points."],
  "plot_suggestion": {
    "recommended": true,
    "plot_type": "bar",
    "x": "Name",
    "y": "Quantity",
    "title": "Inventory Levels of Finished Goods at Risk of Stockout"
  }
}

```

9 Output Evaluation

9.1 Strength

- A SQL table suitable for implementing the ML part
- Consistent JSON output ## Weakness
- Just like in Part D, the NL-SQL interface has not capacity to implement the ML needed
- Add `SUN()` to **pi.Quantity AS OnHand** because a product can exist in multiple inventory location
- `ci.OnHand + rt.TotalSold < ci.ReorderPoint` is logically weak. If `TotalSold` represents demand/consumption, adding it to on-hand inventory makes little business sense; a better simple proxy is to compare on-hand vs recent demand.
- It does not clearly separate a modelling table from a final risk label.

```

[107]: user_content9 = f"""
Database: AdventureWorks
Task: Convert the following natural language request into SQL Server SQL.
Schema summary:
{database_schema}

Example:

Input:
Build a product-level modelling table for inventory risk / stockout prediction.
Return one row per finished good with:
- ProductID
- product name or descriptor
- current total on-hand quantity
- reorder point
- recent demand signal from transaction history
- transaction count
- average transaction quantity
- stockout_risk_flag

```

Output:

```
{{
  "question_understanding": "Build a product-level modelling table for
  ↪predicting which finished goods are at risk of stockout using current
  ↪inventory and recent transaction history.",
  "assumptions": [
    "Only finished goods are included.",
    "Recent demand is approximated using transactions from the last 30 days.",
    "Current on-hand inventory should be summed across all inventory locations.
    ↪",
    "A simple stockout-risk proxy is used in this first version rather than a
    ↪full forecasting model."
  ],
  "sql": "WITH RecentTransactions AS ( SELECT th.ProductID, SUM(th.Quantity) AS
  ↪TotalRecentDemand, COUNT(*) AS TransactionCount, AVG(CAST(th.Quantity AS
  ↪float)) AS AvgTransactionQty FROM Production.TransactionHistory AS th WHERE
  ↪th.TransactionDate >= DATEADD(DAY, -30, GETDATE()) GROUP BY th.ProductID ),
  ↪CurrentInventory AS ( SELECT p.ProductID, p.Name AS ProductName, SUM(pi.
  ↪Quantity) AS TotalOnHand, p.ReorderPoint FROM Production.Product AS p JOIN
  ↪Production.ProductInventory AS pi ON p.ProductID = pi.ProductID WHERE p.
  ↪FinishedGoodsFlag = 1 GROUP BY p.ProductID, p.Name, p.ReorderPoint ) SELECT
  ↪ci.ProductID, ci.ProductName, ci.TotalOnHand, ci.ReorderPoint, COALESCE(rt.
  ↪TotalRecentDemand, 0) AS TotalRecentDemand, COALESCE(rt.TransactionCount, 0)
  ↪AS TransactionCount, COALESCE(rt.AvgTransactionQty, 0) AS AvgTransactionQty,
  ↪CASE WHEN ci.TotalOnHand <= ci.ReorderPoint THEN 1 WHEN ci.TotalOnHand <
  ↪COALESCE(rt.TotalRecentDemand, 0) THEN 1 ELSE 0 END AS stockout_risk_flag
  ↪FROM CurrentInventory AS ci LEFT JOIN RecentTransactions AS rt ON ci.
  ↪ProductID = rt.ProductID ORDER BY stockout_risk_flag DESC, ci.ProductName;",
  "explanation": "The query creates a finished-goods inventory risk table by
  ↪aggregating on-hand stock across all locations and combining it with recent
  ↪transaction-based demand features from the last 30 days. It then applies a
  ↪simple stockout-risk rule based on reorder point and recent demand.",
  "result_shape": "One row per finished good containing ProductID, ProductName,
  ↪TotalOnHand, ReorderPoint, TotalRecentDemand, TransactionCount,
  ↪AvgTransactionQty, and stockout_risk_flag.",
  "validation_checks": [
    "Verify that inventory is summed across all locations for each product.",
    "Verify that only finished goods are included.",
    "Check that products with no recent transactions still appear with
    ↪zero-filled demand features.",
    "Check that stockout_risk_flag is 1 only when the rule conditions are met."
  ],
  "plot_suggestion": {{
    "recommended": true,
    "plot_type": "bar",
    "x": "ProductName",

```

```

        "y": "TotalOnHand",
        "title": "Finished Goods Inventory and Stockout Risk"
    }}
}}

---

Business context:
- This database represents a manufacturing company.
- The goal is to support inventory risk analysis for finished goods.

Important table hints:
- Inventory: Production.ProductInventory + Production.Product + Production.
  ↳ Location
- Transactions: Production.TransactionHistory + Production.Product
- Stockout modelling table: Production.ProductInventory + Production.
  ↳ TransactionHistory + Production.Product

Natural language request:
{user_question5}

Output requirements:
- Return valid JSON only.
"""

```

```

[108]: message5 = [
    {
        "role": "system",
        "content": system
    },
    {
        "role": "user",
        "content": user_content9    # MUST be a string
    }
]
message5

```

```

[108]: [{'role': 'system',
        'content': 'You are an expert data analyst and SQL generator for the
AdventureWorks database.\n\nYour task is to translate a user's natural language
manufacturing or business question into a safe, correct SQL Server query and a
brief explanation.\n\nRules:\n1. Only generate read-only SQL.\n2. Allowed query
types are:\n    - SELECT\n    - Common Table Expressions (CTEs)\n    - ORDER BY\n    - GROUP BY\n    - HAVING\n    - JOINS\n    - window functions\n    - recursive CTEs
if needed\n3. Do not generate:\n    - INSERT\n    - UPDATE\n    - DELETE\n    -
DROP\n    - ALTER\n    - TRUNCATE\n    - EXEC\n    - MERGE\n    - stored
procedures\n4. The SQL dialect is Microsoft SQL Server (AdventureWorks).\n5.

```


Prefer fully qualified table names, for example:\n Production.WorkOrder\n Production.Product\n Production.ProductInventory\n6. Use TOP for row limits, not LIMIT.\n7. If the question is ambiguous, make the smallest reasonable assumption and state it briefly.\n8. If the question cannot be answered from the provided schema, say so clearly.\n9. Use joins only where supported by the schema provided.\n10. Return output in valid JSON only.\n\nYou must return exactly this JSON structure:\n\n{\n "question_understanding": "short restatement of the task",\n "assumptions": ["assumption 1", "assumption 2"],\n "sql": "single SQL Server query as a string",\n "explanation": "brief explanation of how the query works",\n "result_shape": "what the output table should contain",\n "validation_checks": ["check 1", "check 2"],\n "plot_suggestion": {\n "recommended": true,\n "plot_type": "bar|line|pie|none",\n "x": "column name or null",\n "y": "column name or null",\n "title": "plot title or null"\n }\n}\n\nAdditional requirements:\n- The SQL must be executable as written.\n- Do not wrap SQL in markdown fences.\n- If no chart is appropriate, set plot_suggestion.plot_type to "none" and other plot fields to null.\n- If aggregation is used, ensure all non-aggregated selected columns are included in GROUP BY.\n- If ranking is requested, prefer ORDER BY and TOP, or window functions where appropriate.\n- If multi-level bill-of-materials logic is requested, use a recursive CTE.'},\n {'role': 'user',\n 'content': '\nDatabase: AdventureWorks\nTask: Convert the following natural language request into SQL Server SQL.\nSchema summary:\nProduction.WorkOrder\n- WorkOrderID (PK)\n- ProductID (FK -> Production.Product.ProductID)\n- OrderQty\n- StockedQty\n- ScrappedQty\n- StartDate\n- EndDate\n- DueDate\n- ScrapReasonID\n\nProduction.Product\n- ProductID (PK)\n- Name\n- ProductNumber\n- MakeFlag\n- FinishedGoodsFlag\n- Color\n- SafetyStockLevel\n- ReorderPoint\n- StandardCost\n- ListPrice\n\nProduction.ProductInventory\n- ProductID (FK -> Production.Product.ProductID)\n- LocationID (FK -> Production.Location.LocationID)\n- Shelf\n- Bin\n- Quantity\n\nProduction.Location\n- LocationID (PK)\n- Name\n- CostRate\n- Availability\n\nProduction.WorkOrderRouting\n- WorkOrderID (FK -> Production.WorkOrder.WorkOrderID)\n- ProductID\n- OperationSequence\n- LocationID (FK -> Production.Location.LocationID)\n- ScheduledStartDate\n- ScheduledEndDate\n- ActualStartDate\n- ActualEndDate\n- PlannedCost\n- ActualCost\n\nProduction.BillOfMaterials\n- BillOfMaterialsID (PK)\n- ProductAssemblyID\n- ComponentID\n- StartDate\n- EndDate\n- UnitMeasureCode\n- BOMLevel\n- PerAssemblyQty\n\nProduction.TransactionHistory\n- TransactionID\n- ProductID\n- ReferenceOrderID\n- ReferenceOrderLineID\n- TransactionDate\n- TransactionType\n- Quantity\n- ActualCost\n\nExample:\n\nInput:\nBuild a product-level modelling table for inventory risk / stockout prediction.\nReturn one row per finished good with:\n- ProductID\n- product name or descriptor\n- current total on-hand quantity\n- reorder point\n- recent demand signal from transaction history\n- transaction count\n- average transaction quantity\n- stockout_risk_flag\n\nOutput:\n{\n "question_understanding": "Build a product-level modelling table for predicting which finished goods are at risk of

```

stockout using current inventory and recent transaction history.",\n
"assumptions": [\n    "Only finished goods are included.",\n    "Recent demand is approximated using transactions from the last 30 days.",\n    "Current on-hand inventory should be summed across all inventory locations.",\n    "A simple stockout-risk proxy is used in this first version rather than a full forecasting model."],\n    "sql": "WITH RecentTransactions AS ( SELECT th.ProductID, SUM(th.Quantity) AS TotalRecentDemand, COUNT(*) AS TransactionCount, AVG(CAST(th.Quantity AS float)) AS AvgTransactionQty FROM Production.TransactionHistory AS th WHERE th.TransactionDate >= DATEADD(DAY, -30, GETDATE())) GROUP BY th.ProductID ), CurrentInventory AS ( SELECT p.ProductID, p.Name AS ProductName, SUM(pi.Quantity) AS TotalOnHand, p.ReorderPoint FROM Production.Product AS p JOIN Production.ProductInventory AS pi ON p.ProductID = pi.ProductID WHERE p.FinishedGoodsFlag = 1 GROUP BY p.ProductID, p.Name, p.ReorderPoint ) SELECT ci.ProductID, ci.ProductName, ci.TotalOnHand, ci.ReorderPoint, COALESCE(rt.TotalRecentDemand, 0) AS TotalRecentDemand, COALESCE(rt.TransactionCount, 0) AS TransactionCount, COALESCE(rt.AvgTransactionQty, 0) AS AvgTransactionQty, CASE WHEN ci.TotalOnHand <= ci.ReorderPoint THEN 1 WHEN ci.TotalOnHand < COALESCE(rt.TotalRecentDemand, 0) THEN 1 ELSE 0 END AS stockout_risk_flag FROM CurrentInventory AS ci LEFT JOIN RecentTransactions AS rt ON ci.ProductID = rt.ProductID ORDER BY stockout_risk_flag DESC, ci.ProductName;",\n    "explanation": "The query creates a finished-goods inventory risk table by aggregating on-hand stock across all locations and combining it with recent transaction-based demand features from the last 30 days. It then applies a simple stockout-risk rule based on reorder point and recent demand.",\n    "result_shape": "One row per finished good containing ProductID, ProductName, TotalOnHand, ReorderPoint, TotalRecentDemand, TransactionCount, AvgTransactionQty, and stockout_risk_flag.",\n    "validation_checks": [\n        "Verify that inventory is summed across all locations for each product.",\n        "Verify that only finished goods are included.",\n        "Check that products with no recent transactions still appear with zero-filled demand features.",\n        "Check that stockout_risk_flag is 1 only when the rule conditions are met."],\n    "plot_suggestion": {\n        "recommended": true,\n        "plot_type": "bar",\n        "x": "ProductName",\n        "y": "TotalOnHand",\n        "title": "Finished Goods Inventory and Stockout Risk"\n    }\n}\n\n---\n\nBusiness context:\n- This database represents a manufacturing company.\n- The goal is to support inventory risk analysis for finished goods.\n\nImportant table hints:\n- Inventory: Production.ProductInventory + Production.Product + Production.Location\n- Transactions: Production.TransactionHistory + Production.Product\n- Stockout modelling table: Production.ProductInventory + Production.TransactionHistory + Production.Product\n\nNatural language request:\n\nForecast which finished goods will be at risk of stockout given recent transaction history and current on-hand. Approach:\n• Build consumption signals from TransactionHistory (sales/work order issues)\n• Simple baseline: moving-average demand + reorder-point logic\n• ML: classification of "stockout next N days" using lag features\n\nOutput requirements:\n- Return valid JSON only.\n'}]

```

```
[109]: response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=message5 # correct
)
print(response.choices[0].message.content)

{
  "question_understanding": "Forecast which finished goods will be at risk of
stockout using recent transaction history and current on-hand inventory.",
  "assumptions": [
    "Only finished goods are included in the analysis.",
    "Recent demand is approximated using transactions from the last 30 days.",
    "Current on-hand inventory should be summed across all inventory
locations.",
    "A moving average over the past 30 days is used to estimate average demand."
  ],
  "sql": "WITH RecentDemand AS ( SELECT th.ProductID, AVG(th.Quantity) AS
MovingAvgDemand FROM Production.TransactionHistory AS th WHERE
th.TransactionDate >= DATEADD(DAY, -30, GETDATE()) GROUP BY th.ProductID ),
CurrentInventory AS ( SELECT p.ProductID, p.Name AS ProductName,
SUM(pi.Quantity) AS TotalOnHand, p.ReorderPoint FROM Production.Product AS p
JOIN Production.ProductInventory AS pi ON p.ProductID = pi.ProductID WHERE
p.FinishedGoodsFlag = 1 GROUP BY p.ProductID, p.Name, p.ReorderPoint ) SELECT
ci.ProductID, ci.ProductName, ci.TotalOnHand, ci.ReorderPoint,
COALESCE(rd.MovingAvgDemand, 0) AS MovingAvgDemand, CASE WHEN ci.TotalOnHand <=
ci.ReorderPoint THEN 1 WHEN ci.TotalOnHand < COALESCE(rd.MovingAvgDemand, 0)
THEN 1 ELSE 0 END AS stockout_risk_flag FROM CurrentInventory AS ci LEFT JOIN
RecentDemand AS rd ON ci.ProductID = rd.ProductID ORDER BY stockout_risk_flag
DESC, ci.ProductName;",
  "explanation": "The query calculates the moving average demand from the last
30 days of transaction history for each finished good and compares it with the
current total on-hand stock and reorder point to flag items at risk of stockout.
The results include product IDs, names, current inventory, reorder points,
moving average demand, and a flag indicating stockout risk.",
  "result_shape": "One row per finished good containing ProductID, ProductName,
TotalOnHand, ReorderPoint, MovingAvgDemand, and stockout_risk_flag.",
  "validation_checks": [
    "Verify that inventory totals are correctly summed across all locations for
each finished good.",
    "Check that only finished goods are included in the results.",
    "Ensure that finished goods with no recent transactions still appear with
zero moving average demand.",
    "Confirm that stockout_risk_flag is correctly set based on the defined
conditions."
  ],
  "plot_suggestion": {
    "recommended": true,
    "plot_type": "bar",

```

```

    "x": "ProductName",
    "y": "TotalOnHand",
    "title": "Finished Goods Inventory and Stockout Risk Analysis"
  }
}

```

```

[110]: query5 = """
WITH RecentDemand AS (
    SELECT
        th.ProductID,
        AVG(th.Quantity) AS MovingAverageDemand
    FROM Production.TransactionHistory AS th
    WHERE th.TransactionDate >= DATEADD(DAY, -30, GETDATE())
    GROUP BY th.ProductID
),
InventoryLevels AS (
    SELECT
        p.ProductID,
        p.Name AS ProductName,
        SUM(pi.Quantity) AS TotalOnHand,
        p.ReorderPoint
    FROM Production.Product AS p
    JOIN Production.ProductInventory AS pi
        ON p.ProductID = pi.ProductID
    WHERE p.FinishedGoodsFlag = 1
    GROUP BY
        p.ProductID,
        p.Name,
        p.ReorderPoint
)
SELECT
    il.ProductID,
    il.ProductName,
    il.TotalOnHand,
    il.ReorderPoint,
    COALESCE(rd.MovingAverageDemand, 0) AS MovingAverageDemand,
    CASE
        WHEN il.TotalOnHand <= il.ReorderPoint THEN 1
        WHEN il.TotalOnHand < COALESCE(rd.MovingAverageDemand, 0) THEN 1
        ELSE 0
    END AS stockout_risk_flag
FROM InventoryLevels AS il
LEFT JOIN RecentDemand AS rd
    ON il.ProductID = rd.ProductID
ORDER BY
    stockout_risk_flag DESC;
"""

```

```
df_stockout = pd.read_sql(query5, conn)
df_stockout
```

C:\Users\emper\AppData\Local\Temp\ipykernel_55500\86143689.py:43: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
df_stockout = pd.read_sql(query5, conn)
```

```
[110]:
```

	ProductID	ProductName	TotalOnHand	ReorderPoint	\
0	910	HL Mountain Seat/Saddle	355	375	
1	956	Touring-1000 Yellow, 54	75	75	
2	859	Half-Finger Gloves, M	0	3	
3	853	Women's Tights, M	0	3	
4	876	Hitch Rack - 4-Bike	0	3	
..	...	...	...	...	
218	848	Headlights - Weatherproof	216	3	
219	849	Men's Sports Shorts, M	108	3	
220	850	Men's Sports Shorts, L	144	3	
221	851	Men's Sports Shorts, XL	180	3	
222	852	Women's Tights, S	324	3	

	MovingAverageDemand	stockout_risk_flag
0	0	1
1	0	1
2	0	1
3	0	1
4	0	1
..	...	...
218	0	0
219	0	0
220	0	0
221	0	0
222	0	0

[223 rows x 6 columns]

```
[111]: #df_stockout.to_csv("Stockout_df.csv", index=False)
```

```
[164]: stockout = pd.read_csv("Stockout_df.csv")
stockout
```

```
[164]:
```

	ProductID	ProductName	TotalOnHand	ReorderPoint	\
0	910	HL Mountain Seat/Saddle	355	375	
1	956	Touring-1000 Yellow, 54	75	75	
2	859	Half-Finger Gloves, M	0	3	
3	853	Women's Tights, M	0	3	

4	876	Hitch Rack - 4-Bike	0	3
..	...	...	...	...
218	848	Headlights - Weatherproof	216	3
219	849	Men's Sports Shorts, M	108	3
220	850	Men's Sports Shorts, L	144	3
221	851	Men's Sports Shorts, XL	180	3
222	852	Women's Tights, S	324	3

	MovingAverageDemand	stockout_risk_flag
0	0	1
1	0	1
2	0	1
3	0	1
4	0	1
..	...	...
218	0	0
219	0	0
220	0	0
221	0	0
222	0	0

[223 rows x 6 columns]

```
[212]: stockout.stockout_risk_flag.value_counts()
```

```
[212]: stockout_risk_flag
0      217
1         6
Name: count, dtype: int64
```

```
[ ]:
```